

Problem Solving using Python

Assignment - 1 : 90 Solved Answers

Course Code: 25CSE144 | Sem/Sec: I/C | Max. Marks: 5M

Department of Computer Science & Engineering, New Horizon College of Engineering

Groups	3 groups (roll number ends in 0,3,5,7 / 1,6,8 / 2,4,9)
Questions	9 questions (3 per group)
Answers	10 different solved sets per question = 90 answers
Every answer has	Explanation, program with its own variables, input and output statements, and the real output captured by running the program. Algorithm and flowchart are included wherever the question asks.
How to choose	Pick the set whose number matches the last digit of your roll number (digit 0 = Set 10) so every student submits a different answer.

Contents

Section 1 : Group 1 (Roll Number ends in 0, 3, 5, 7) - Question 1 (Answers 1 - 10)	2
Section 2 : Group 1 (Roll Number ends in 0, 3, 5, 7) - Question 2 (Answers 11 - 20)	15
Section 3 : Group 1 (Roll Number ends in 0, 3, 5, 7) - Question 3 (Answers 21 - 30)	21
Section 4 : Group 2 (Roll Number ends in 1, 6, 8) - Question 1 (Answers 31 - 40)	32
Section 5 : Group 2 (Roll Number ends in 1, 6, 8) - Question 2 (Answers 41 - 50)	40
Section 6 : Group 2 (Roll Number ends in 1, 6, 8) - Question 3 (Answers 51 - 60)	54
Section 7 : Group 3 (Roll Number ends in 2, 4, 9) - Question 1 (Answers 61 - 70)	65
Section 8 : Group 3 (Roll Number ends in 2, 4, 9) - Question 2 (Answers 71 - 80)	75
Section 9 : Group 3 (Roll Number ends in 2, 4, 9) - Question 3 (Answers 81 - 90)	89

Section 1 : Group 1 (Roll Number ends in 0, 3, 5, 7) - Question 1 (Answers 1 - 10)

Question: Explain Arithmetic and Comparison operators with example. A banking application calculates the total balance after adding a deposit and subtracting a withdrawal. Identify the arithmetic operators required for these operations.

Answer 1 - Set 1 of 10 (Banking - Ravi Kumar)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values a = 17 and b = 5.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	17 + 5	22
-	Subtraction	17 - 5	12
*	Multiplication	17 * 5	85
/	Division (float result)	17 / 5	3.4
//	Floor division (integer quotient)	17 // 5	3
%	Modulus (remainder)	17 % 5	2
**	Exponent (power)	17 ** 5	1419857

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	17 == 5	False
!=	Not equal to	17 != 5	True
>	Greater than	17 > 5	True
<	Less than	17 < 5	False
>=	Greater than or equal to	17 >= 5	True
<=	Less than or equal to	17 <= 5	False

Banking application: the total balance is found with the formula **total_balance = balance + deposit - withdrawal**. The arithmetic operators required are the **addition (+)** operator to add the deposit (5000) and the **subtraction (-)** operator to subtract the withdrawal (2000). For this account the total is 15000 + 5000 - 2000 = 18000.

Program

```
# Part A : arithmetic and comparison operators
a = int(input("Enter value of a: "))
b = int(input("Enter value of b: "))
print("a + b =", a + b)
print("a - b =", a - b)
print("a * b =", a * b)
print("a / b =", a / b)
print("a // b =", a // b)
print("a % b =", a % b)
print("a ** b =", a ** b)
print("a == b :", a == b)
print("a != b :", a != b)
print("a > b :", a > b)
print("a < b :", a < b)
print("a >= b :", a >= b)
print("a <= b :", a <= b)
# Part B : banking application
name = input("Enter account holder name: ")
balance = float(input("Enter current balance: "))
deposit = float(input("Enter deposit amount: "))
withdrawal = float(input("Enter withdrawal amount: "))
total_balance = balance + deposit - withdrawal # '+' adds the deposit, '-' subtracts the withdrawal
print("Account Holder :", name)
print("Total Balance :", total_balance)
```

```
print("Withdrawal possible (withdrawal <= balance + deposit):", withdrawal <= balance + deposit)
print("Balance is at least 1000:", total_balance >= 1000)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of a: 17
Enter value of b: 5
a + b = 22
a - b = 12
a * b = 85
a / b = 3.4
a // b = 3
a % b = 2
a ** b = 1419857
a == b : False
a != b : True
a > b : True
a < b : False
a >= b : True
a <= b : False
Enter account holder name: Ravi Kumar
Enter current balance: 15000
Enter deposit amount: 5000
Enter withdrawal amount: 2000
Account Holder : Ravi Kumar
Total Balance : 18000.0
Withdrawal possible (withdrawal <= balance + deposit): True
Balance is at least 1000: True
```

Answer 2 - Set 2 of 10 (Banking - Anita Sharma)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values $x = 23$ and $y = 4$.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	$23 + 4$	27
-	Subtraction	$23 - 4$	19
*	Multiplication	$23 * 4$	92
/	Division (float result)	$23 / 4$	5.75
//	Floor division (integer quotient)	$23 // 4$	5
%	Modulus (remainder)	$23 \% 4$	3
**	Exponent (power)	$23 ** 4$	279841

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	$23 == 4$	False
!=	Not equal to	$23 != 4$	True
>	Greater than	$23 > 4$	True
<	Less than	$23 < 4$	False
>=	Greater than or equal to	$23 >= 4$	True
<=	Less than or equal to	$23 <= 4$	False

Banking application: the total balance is found with the formula **closing_bal = opening_bal + credit - debit**. The arithmetic operators required are the **addition (+)** operator to add the deposit (7500) and the **subtraction (-)** operator to subtract the withdrawal (3000). For this account the total is $28500 + 7500 - 3000 = 33000$.

Program

```
# Part A : arithmetic and comparison operators
x = int(input("Enter value of x: "))
y = int(input("Enter value of y: "))
print(f"{x} + {y} ->", x + y)
print(f"{x} - {y} ->", x - y)
print(f"{x} * {y} ->", x * y)
print(f"{x} / {y} ->", x / y)
print(f"{x} // {y} ->", x // y)
print(f"{x} % {y} ->", x % y)
print(f"{x} ** {y} ->", x ** y)
print(f"Is {x} == {y}?", x == y)
print(f"Is {x} != {y}?", x != y)
print(f"Is {x} > {y}?", x > y)
print(f"Is {x} < {y}?", x < y)
print(f"Is {x} >= {y}?", x >= y)
print(f"Is {x} <= {y}?", x <= y)
# Part B : banking application
name = input("Enter account holder name: ")
opening_bal = float(input("Enter current balance: "))
credit = float(input("Enter deposit amount: "))
debit = float(input("Enter withdrawal amount: "))
closing_bal = opening_bal + credit - debit # '+' adds the deposit, '-' subtracts the withdrawal
print(f"Dear {name}, your balance is Rs. {closing_bal:.2f}")
print("Withdrawal possible (debit <= opening_bal + credit):", debit <= opening_bal + credit)
print("Balance is at least 500:", closing_bal >= 500)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of x: 23
Enter value of y: 4
23 + 4 -> 27
23 - 4 -> 19
23 * 4 -> 92
23 / 4 -> 5.75
23 // 4 -> 5
23 % 4 -> 3
23 ** 4 -> 279841
Is 23 == 4? False
Is 23 != 4? True
Is 23 > 4? True
Is 23 < 4? False
Is 23 >= 4? True
Is 23 <= 4? False
Enter account holder name: Anita Sharma
Enter current balance: 28500
Enter deposit amount: 7500
Enter withdrawal amount: 3000
Dear Anita Sharma, your balance is Rs. 33000.00
Withdrawal possible (debit <= opening_bal + credit): True
Balance is at least 500: True
```

Answer 3 - Set 3 of 10 (Banking - Mohan Das)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values $n1 = 30$ and $n2 = 8$.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	$30 + 8$	38
-	Subtraction	$30 - 8$	22
*	Multiplication	$30 * 8$	240
/	Division (float result)	$30 / 8$	3.75
//	Floor division (integer quotient)	$30 // 8$	3
%	Modulus (remainder)	$30 \% 8$	6
**	Exponent (power)	$30 ** 8$	656100000000

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	30 == 8	False
!=	Not equal to	30 != 8	True
>	Greater than	30 > 8	True
<	Less than	30 < 8	False
>=	Greater than or equal to	30 >= 8	True
<=	Less than or equal to	30 <= 8	False

Banking application: the total balance is found with the formula **new_bal = acc_bal + amount_added - amount_taken**. The arithmetic operators required are the **addition (+)** operator to add the deposit (10000) and the **subtraction (-)** operator to subtract the withdrawal (4500). For this account the total is 42000 + 10000 - 4500 = 47500.

Program

```
# Part A : arithmetic and comparison operators
n1 = int(input("Enter value of n1: "))
n2 = int(input("Enter value of n2: "))
print("n1 + n2 =", n1 + n2)
print("n1 - n2 =", n1 - n2)
print("n1 * n2 =", n1 * n2)
print("n1 / n2 =", n1 / n2)
print("n1 // n2 =", n1 // n2)
print("n1 % n2 =", n1 % n2)
print("n1 ** n2 =", n1 ** n2)
print("n1 == n2 :", n1 == n2)
print("n1 != n2 :", n1 != n2)
print("n1 > n2 :", n1 > n2)
print("n1 < n2 :", n1 < n2)
print("n1 >= n2 :", n1 >= n2)
print("n1 <= n2 :", n1 <= n2)
# Part B : banking application
name = input("Enter account holder name: ")
acc_bal = float(input("Enter current balance: "))
amount_added = float(input("Enter deposit amount: "))
amount_taken = float(input("Enter withdrawal amount: "))
new_bal = acc_bal + amount_added - amount_taken # '+' adds the deposit, '-' subtracts the withdrawal
print("Hello " + name + "! Closing balance = " + str(new_bal))
print("Withdrawal possible (amount_taken <= acc_bal + amount_added):", amount_taken <= acc_bal + amount_added)
print("Balance is at least 2000:", new_bal >= 2000)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of n1: 30
Enter value of n2: 8
n1 + n2 = 38
n1 - n2 = 22
n1 * n2 = 240
n1 / n2 = 3.75
n1 // n2 = 3
n1 % n2 = 6
n1 ** n2 = 656100000000
n1 == n2 : False
n1 != n2 : True
n1 > n2 : True
n1 < n2 : False
n1 >= n2 : True
n1 <= n2 : False
Enter account holder name: Mohan Das
Enter current balance: 42000
Enter deposit amount: 10000
Enter withdrawal amount: 4500
Hello Mohan Das! Closing balance = 47500.0
Withdrawal possible (amount_taken <= acc_bal + amount_added): True
Balance is at least 2000: True
```

Answer 4 - Set 4 of 10 (Banking - Priya Nair)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values $p = 45$ and $q = 6$.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	$45 + 6$	51
-	Subtraction	$45 - 6$	39
*	Multiplication	$45 * 6$	270
/	Division (float result)	$45 / 6$	7.5
//	Floor division (integer quotient)	$45 // 6$	7
%	Modulus (remainder)	$45 \% 6$	3
**	Exponent (power)	$45 ** 6$	8303765625

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	$45 == 6$	False
!=	Not equal to	$45 != 6$	True
>	Greater than	$45 > 6$	True
<	Less than	$45 < 6$	False
>=	Greater than or equal to	$45 >= 6$	True
<=	Less than or equal to	$45 <= 6$	False

Banking application: the total balance is found with the formula **final_amt = savings + cash_in - cash_out**. The arithmetic operators required are the **addition (+)** operator to add the deposit (2200) and the **subtraction (-)** operator to subtract the withdrawal (1500). For this account the total is $9800 + 2200 - 1500 = 10500$.

Program

```
# Part A : arithmetic and comparison operators
p = int(input("Enter value of p: "))
q = int(input("Enter value of q: "))
print(f"{p} + {q} ->", p + q)
print(f"{p} - {q} ->", p - q)
print(f"{p} * {q} ->", p * q)
print(f"{p} / {q} ->", p / q)
print(f"{p} // {q} ->", p // q)
print(f"{p} % {q} ->", p % q)
print(f"{p} ** {q} ->", p ** q)
print(f"Is {p} == {q}?", p == q)
print(f"Is {p} != {q}?", p != q)
print(f"Is {p} > {q}?", p > q)
print(f"Is {p} < {q}?", p < q)
print(f"Is {p} >= {q}?", p >= q)
print(f"Is {p} <= {q}?", p <= q)
# Part B : banking application
name = input("Enter account holder name: ")
savings = float(input("Enter current balance: "))
cash_in = float(input("Enter deposit amount: "))
cash_out = float(input("Enter withdrawal amount: "))
final_amt = savings + cash_in - cash_out # '+' adds the deposit, '-' subtracts the withdrawal
print("Account Holder :", name)
print("Total Balance :", final_amt)
print("Withdrawal possible (cash_out <= savings + cash_in):", cash_out <= savings + cash_in)
print("Balance is at least 1000:", final_amt >= 1000)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of p: 45
Enter value of q: 6
45 + 6 -> 51
45 - 6 -> 39
45 * 6 -> 270
45 / 6 -> 7.5
```

```

45 // 6 -> 7
45 % 6 -> 3
45 ** 6 -> 8303765625
Is 45 == 6? False
Is 45 != 6? True
Is 45 > 6? True
Is 45 < 6? False
Is 45 >= 6? True
Is 45 <= 6? False
Enter account holder name: Priya Nair
Enter current balance: 9800
Enter deposit amount: 2200
Enter withdrawal amount: 1500
Account Holder : Priya Nair
Total Balance : 10500.0
Withdrawal possible (cash_out <= savings + cash_in): True
Balance is at least 1000: True

```

Answer 5 - Set 5 of 10 (Banking - Suresh Reddy)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values $m = 19$ and $n = 4$.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	$19 + 4$	23
-	Subtraction	$19 - 4$	15
*	Multiplication	$19 * 4$	76
/	Division (float result)	$19 / 4$	4.75
//	Floor division (integer quotient)	$19 // 4$	4
%	Modulus (remainder)	$19 \% 4$	3
**	Exponent (power)	$19 ** 4$	130321

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	$19 == 4$	False
!=	Not equal to	$19 != 4$	True
>	Greater than	$19 > 4$	True
<	Less than	$19 < 4$	False
>=	Greater than or equal to	$19 >= 4$	True
<=	Less than or equal to	$19 <= 4$	False

Banking application: the total balance is found with the formula **net_balance = current_amt + deposit_amt - withdraw_amt**. The arithmetic operators required are the **addition (+)** operator to add the deposit (12500) and the **subtraction (-)** operator to subtract the withdrawal (9000). For this account the total is $67250 + 12500 - 9000 = 70750$.

Program

```

# Part A : arithmetic and comparison operators
m = int(input("Enter value of m: "))
n = int(input("Enter value of n: "))
print("m + n =", m + n)
print("m - n =", m - n)
print("m * n =", m * n)
print("m / n =", m / n)
print("m // n =", m // n)
print("m % n =", m % n)
print("m ** n =", m ** n)
print("m == n :", m == n)
print("m != n :", m != n)
print("m > n :", m > n)
print("m < n :", m < n)
print("m >= n :", m >= n)
print("m <= n :", m <= n)

```

```
# Part B : banking application
name = input("Enter account holder name: ")
current_amt = float(input("Enter current balance: "))
deposit_amt = float(input("Enter deposit amount: "))
withdraw_amt = float(input("Enter withdrawal amount: "))
net_balance = current_amt + deposit_amt - withdraw_amt # '+' adds the deposit, '-' subtracts the withdrawal
print(f"Dear {name}, your balance is Rs. {net_balance:.2f}")
print("Withdrawal possible (withdraw_amt <= current_amt + deposit_amt):", withdraw_amt <= current_amt + deposit_amt)
print("Balance is at least 5000:", net_balance >= 5000)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of m: 19
Enter value of n: 4
m + n = 23
m - n = 15
m * n = 76
m / n = 4.75
m // n = 4
m % n = 3
m ** n = 130321
m == n : False
m != n : True
m > n : True
m < n : False
m >= n : True
m <= n : False
Enter account holder name: Suresh Reddy
Enter current balance: 67250
Enter deposit amount: 12500
Enter withdrawal amount: 9000
Dear Suresh Reddy, your balance is Rs. 70750.00
Withdrawal possible (withdraw_amt <= current_amt + deposit_amt): True
Balance is at least 5000: True
```

Answer 6 - Set 6 of 10 (Banking - Kavya Rao)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values num1 = 52 and num2 = 8.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	52 + 8	60
-	Subtraction	52 - 8	44
*	Multiplication	52 * 8	416
/	Division (float result)	52 / 8	6.5
//	Floor division (integer quotient)	52 // 8	6
%	Modulus (remainder)	52 % 8	4
**	Exponent (power)	52 ** 8	53459728531456

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	52 == 8	False
!=	Not equal to	52 != 8	True
>	Greater than	52 > 8	True
<	Less than	52 < 8	False
>=	Greater than or equal to	52 >= 8	True
<=	Less than or equal to	52 <= 8	False

Banking application: the total balance is found with the formula **total = bal + dep - wd**. The arithmetic operators required are the **addition (+)** operator to add the deposit (4500) and the **subtraction (-)** operator to subtract the withdrawal (6000). For this account the total is 31500 + 4500 - 6000 = 30000.

Program

```
# Part A : arithmetic and comparison operators
num1 = int(input("Enter value of num1: "))
num2 = int(input("Enter value of num2: "))
print(f"{num1} + {num2} ->", num1 + num2)
print(f"{num1} - {num2} ->", num1 - num2)
print(f"{num1} * {num2} ->", num1 * num2)
print(f"{num1} / {num2} ->", num1 / num2)
print(f"{num1} // {num2} ->", num1 // num2)
print(f"{num1} % {num2} ->", num1 % num2)
print(f"{num1} ** {num2} ->", num1 ** num2)
print(f"Is {num1} == {num2}?", num1 == num2)
print(f"Is {num1} != {num2}?", num1 != num2)
print(f"Is {num1} > {num2}?", num1 > num2)
print(f"Is {num1} < {num2}?", num1 < num2)
print(f"Is {num1} >= {num2}?", num1 >= num2)
print(f"Is {num1} <= {num2}?", num1 <= num2)
# Part B : banking application
name = input("Enter account holder name: ")
bal = float(input("Enter current balance: "))
dep = float(input("Enter deposit amount: "))
wd = float(input("Enter withdrawal amount: "))
total = bal + dep - wd      # '+' adds the deposit, '-' subtracts the withdrawal
print("Hello " + name + "! Closing balance = " + str(total))
print("Withdrawal possible (wd <= bal + dep):", wd <= bal + dep)
print("Balance is at least 500:", total >= 500)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of num1: 52
Enter value of num2: 8
52 + 8 -> 60
52 - 8 -> 44
52 * 8 -> 416
52 / 8 -> 6.5
52 // 8 -> 6
52 % 8 -> 4
52 ** 8 -> 53459728531456
Is 52 == 8? False
Is 52 != 8? True
Is 52 > 8? True
Is 52 < 8? False
Is 52 >= 8? True
Is 52 <= 8? False
Enter account holder name: Kavya Rao
Enter current balance: 31500
Enter deposit amount: 4500
Enter withdrawal amount: 6000
Hello Kavya Rao! Closing balance = 30000.0
Withdrawal possible (wd <= bal + dep): True
Balance is at least 500: True
```

Answer 7 - Set 7 of 10 (Banking - Imran Khan)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values first = 27 and second = 6.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	27 + 6	33
-	Subtraction	27 - 6	21
*	Multiplication	27 * 6	162
/	Division (float result)	27 / 6	4.5
//	Floor division (integer quotient)	27 // 6	4
%	Modulus (remainder)	27 % 6	3
**	Exponent (power)	27 ** 6	387420489

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	27 == 6	False
!=	Not equal to	27 != 6	True
>	Greater than	27 > 6	True
<	Less than	27 < 6	False
>=	Greater than or equal to	27 >= 6	True
<=	Less than or equal to	27 <= 6	False

Banking application: the total balance is found with the formula **updated_balance = old_balance + money_in - money_out**. The arithmetic operators required are the **addition (+)** operator to add the deposit (3250) and the **subtraction (-)** operator to subtract the withdrawal (2250). For this account the total is 12750 + 3250 - 2250 = 13750.

Program

```
# Part A : arithmetic and comparison operators
first = int(input("Enter value of first: "))
second = int(input("Enter value of second: "))
print("first + second =", first + second)
print("first - second =", first - second)
print("first * second =", first * second)
print("first / second =", first / second)
print("first // second =", first // second)
print("first % second =", first % second)
print("first ** second =", first ** second)
print("first == second :", first == second)
print("first != second :", first != second)
print("first > second :", first > second)
print("first < second :", first < second)
print("first >= second :", first >= second)
print("first <= second :", first <= second)
# Part B : banking application
name = input("Enter account holder name: ")
old_balance = float(input("Enter current balance: "))
money_in = float(input("Enter deposit amount: "))
money_out = float(input("Enter withdrawal amount: "))
updated_balance = old_balance + money_in - money_out      # '+' adds the deposit, '-' subtracts the withdrawal
print("Account Holder :", name)
print("Total Balance  :", updated_balance)
print("Withdrawal possible (money_out <= old_balance + money_in):", money_out <= old_balance + money_in)
print("Balance is at least 1500:", updated_balance >= 1500)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of first: 27
Enter value of second: 6
first + second = 33
first - second = 21
first * second = 162
first / second = 4.5
first // second = 4
first % second = 3
first ** second = 387420489
first == second : False
first != second : True
first > second : True
first < second : False
first >= second : True
first <= second : False
Enter account holder name: Imran Khan
Enter current balance: 12750
Enter deposit amount: 3250
Enter withdrawal amount: 2250
Account Holder : Imran Khan
Total Balance  : 13750.0
Withdrawal possible (money_out <= old_balance + money_in): True
Balance is at least 1500: True
```

Answer 8 - Set 8 of 10 (Banking - Deepa Iyer)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values $u = 49$ and $v = 10$.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	$49 + 10$	59
-	Subtraction	$49 - 10$	39
*	Multiplication	$49 * 10$	490
/	Division (float result)	$49 / 10$	4.9
//	Floor division (integer quotient)	$49 // 10$	4
%	Modulus (remainder)	$49 \% 10$	9
**	Exponent (power)	$49 ** 10$	79792266297612001

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	$49 == 10$	False
!=	Not equal to	$49 != 10$	True
>	Greater than	$49 > 10$	True
<	Less than	$49 < 10$	False
>=	Greater than or equal to	$49 >= 10$	True
<=	Less than or equal to	$49 <= 10$	False

Banking application: the total balance is found with the formula $\text{balance_now} = \text{account_bal} + \text{paid_in} - \text{paid_out}$. The arithmetic operators required are the **addition (+)** operator to add the deposit (8000) and the **subtraction (-)** operator to subtract the withdrawal (5500). For this account the total is $54000 + 8000 - 5500 = 56500$.

Program

```
# Part A : arithmetic and comparison operators
u = int(input("Enter value of u: "))
v = int(input("Enter value of v: "))
print(f"{u} + {v} ->", u + v)
print(f"{u} - {v} ->", u - v)
print(f"{u} * {v} ->", u * v)
print(f"{u} / {v} ->", u / v)
print(f"{u} // {v} ->", u // v)
print(f"{u} % {v} ->", u % v)
print(f"{u} ** {v} ->", u ** v)
print(f"Is {u} == {v}?", u == v)
print(f"Is {u} != {v}?", u != v)
print(f"Is {u} > {v}?", u > v)
print(f"Is {u} < {v}?", u < v)
print(f"Is {u} >= {v}?", u >= v)
print(f"Is {u} <= {v}?", u <= v)
# Part B : banking application
name = input("Enter account holder name: ")
account_bal = float(input("Enter current balance: "))
paid_in = float(input("Enter deposit amount: "))
paid_out = float(input("Enter withdrawal amount: "))
balance_now = account_bal + paid_in - paid_out # '+' adds the deposit, '-' subtracts the withdrawal
print(f"Dear {name}, your balance is Rs. {balance_now:.2f}")
print("Withdrawal possible (paid_out <= account_bal + paid_in):", paid_out <= account_bal + paid_in)
print("Balance is at least 1000:", balance_now >= 1000)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of u: 49
Enter value of v: 10
49 + 10 -> 59
49 - 10 -> 39
49 * 10 -> 490
49 / 10 -> 4.9
49 // 10 -> 4
```

```

49 % 10 -> 9
49 ** 10 -> 79792266297612001
Is 49 == 10? False
Is 49 != 10? True
Is 49 > 10? True
Is 49 < 10? False
Is 49 >= 10? True
Is 49 <= 10? False
Enter account holder name: Deepa Iyer
Enter current balance: 54000
Enter deposit amount: 8000
Enter withdrawal amount: 5500
Dear Deepa Iyer, your balance is Rs. 56500.00
Withdrawal possible (paid_out <= account_bal + paid_in): True
Balance is at least 1000: True

```

Answer 9 - Set 9 of 10 (Banking - Arjun Patel)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values $s = 38$ and $t = 5$.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	$38 + 5$	43
-	Subtraction	$38 - 5$	33
*	Multiplication	$38 * 5$	190
/	Division (float result)	$38 / 5$	7.6
//	Floor division (integer quotient)	$38 // 5$	7
%	Modulus (remainder)	$38 \% 5$	3
**	Exponent (power)	$38 ** 5$	79235168

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	$38 == 5$	False
!=	Not equal to	$38 != 5$	True
>	Greater than	$38 > 5$	True
<	Less than	$38 < 5$	False
>=	Greater than or equal to	$38 >= 5$	True
<=	Less than or equal to	$38 <= 5$	False

Banking application: the total balance is found with the formula $\text{end_amt} = \text{start_amt} + \text{add_amt} - \text{sub_amt}$. The arithmetic operators required are the **addition (+)** operator to add the deposit (6600) and the **subtraction (-)** operator to subtract the withdrawal (3400). For this account the total is $23400 + 6600 - 3400 = 26600$.

Program

```

# Part A : arithmetic and comparison operators
s = int(input("Enter value of s: "))
t = int(input("Enter value of t: "))
print("s + t =", s + t)
print("s - t =", s - t)
print("s * t =", s * t)
print("s / t =", s / t)
print("s // t =", s // t)
print("s % t =", s % t)
print("s ** t =", s ** t)
print("s == t :", s == t)
print("s != t :", s != t)
print("s > t :", s > t)
print("s < t :", s < t)
print("s >= t :", s >= t)
print("s <= t :", s <= t)
# Part B : banking application
name = input("Enter account holder name: ")

```

```

start_amt = float(input("Enter current balance: "))
add_amt = float(input("Enter deposit amount: "))
sub_amt = float(input("Enter withdrawal amount: "))
end_amt = start_amt + add_amt - sub_amt      # '+' adds the deposit, '-' subtracts the withdrawal
print("Hello " + name + "! Closing balance = " + str(end_amt))
print("Withdrawal possible (sub_amt <= start_amt + add_amt):", sub_amt <= start_amt + add_amt)
print("Balance is at least 2500:", end_amt >= 2500)

```

Sample Input / Output (typed values appear after each prompt)

```

Enter value of s: 38
Enter value of t: 5
s + t = 43
s - t = 33
s * t = 190
s / t = 7.6
s // t = 7
s % t = 3
s ** t = 79235168
s == t : False
s != t : True
s > t : True
s < t : False
s >= t : True
s <= t : False
Enter account holder name: Arjun Patel
Enter current balance: 23400
Enter deposit amount: 6600
Enter withdrawal amount: 3400
Hello Arjun Patel! Closing balance = 26600.0
Withdrawal possible (sub_amt <= start_amt + add_amt): True
Balance is at least 2500: True

```

Answer 10 - Set 10 of 10 (Banking - Meena Joshi)

Arithmetic operators perform mathematical calculations on numeric operands. **Comparison (relational) operators** compare two values and always return a Boolean result, **True** or **False**. The tables below use the values val1 = 41 and val2 = 8.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	41 + 8	49
-	Subtraction	41 - 8	33
*	Multiplication	41 * 8	328
/	Division (float result)	41 / 8	5.125
//	Floor division (integer quotient)	41 // 8	5
%	Modulus (remainder)	41 % 8	1
**	Exponent (power)	41 ** 8	7984925229121

Comparison operators

Operator	Meaning	Example	Result
==	Equal to	41 == 8	False
!=	Not equal to	41 != 8	True
>	Greater than	41 > 8	True
<	Less than	41 < 8	False
>=	Greater than or equal to	41 >= 8	True
<=	Less than or equal to	41 <= 8	False

Banking application: the total balance is found with the formula **available_balance = principal + deposit_value - withdrawal_value**. The arithmetic operators required are the **addition (+)** operator to add the deposit (15000) and the **subtraction (-)** operator to subtract the withdrawal (10000). For this account the total is 88000 + 15000 - 10000 = 93000.

Program

```
# Part A : arithmetic and comparison operators
val1 = int(input("Enter value of val1: "))
val2 = int(input("Enter value of val2: "))
print(f"{val1} + {val2} ->", val1 + val2)
print(f"{val1} - {val2} ->", val1 - val2)
print(f"{val1} * {val2} ->", val1 * val2)
print(f"{val1} / {val2} ->", val1 / val2)
print(f"{val1} // {val2} ->", val1 // val2)
print(f"{val1} % {val2} ->", val1 % val2)
print(f"{val1} ** {val2} ->", val1 ** val2)
print(f"Is {val1} == {val2}?", val1 == val2)
print(f"Is {val1} != {val2}?", val1 != val2)
print(f"Is {val1} > {val2}?", val1 > val2)
print(f"Is {val1} < {val2}?", val1 < val2)
print(f"Is {val1} >= {val2}?", val1 >= val2)
print(f"Is {val1} <= {val2}?", val1 <= val2)
# Part B : banking application
name = input("Enter account holder name: ")
principal = float(input("Enter current balance: "))
deposit_value = float(input("Enter deposit amount: "))
withdrawal_value = float(input("Enter withdrawal amount: "))
available_balance = principal + deposit_value - withdrawal_value # '+' adds the deposit, '-' subtracts the
print("Account Holder :", name)
print("Total Balance :", available_balance)
print("Withdrawal possible (withdrawal_value <= principal + deposit_value):", withdrawal_value <= principal + de
print("Balance is at least 3000:", available_balance >= 3000)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter value of val1: 41
Enter value of val2: 8
41 + 8 -> 49
41 - 8 -> 33
41 * 8 -> 328
41 / 8 -> 5.125
41 // 8 -> 5
41 % 8 -> 1
41 ** 8 -> 7984925229121
Is 41 == 8? False
Is 41 != 8? True
Is 41 > 8? True
Is 41 < 8? False
Is 41 >= 8? True
Is 41 <= 8? False
Enter account holder name: Meena Joshi
Enter current balance: 88000
Enter deposit amount: 15000
Enter withdrawal amount: 10000
Account Holder : Meena Joshi
Total Balance : 93000.0
Withdrawal possible (withdrawal_value <= principal + deposit_value): True
Balance is at least 3000: True
```

Section 2 : Group 1 (Roll Number ends in 0, 3, 5, 7) - Question 2 (Answers 11 - 20)

Question: Explain the for loop and range() function with a suitable example. A programmer needs to execute a set of statements exactly 10 times. Which Python loop can be used?

Answer 11 - Set 1 of 10 (Multiplication table)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(7)	stop only: starts at 0, step 1	[0, 1, 2, 3, 4, 5, 6]
range(3, 9)	start and stop: step 1	[3, 4, 5, 6, 7, 8]
range(2, 15, 3)	start, stop and step	[2, 5, 8, 11, 14]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(1, 11)**, which runs the body exactly 10 times.

Program

```
num = int(input("Enter a number for its table: "))
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter a number for its table: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Answer 12 - Set 2 of 10 (Greeting printed 10 times)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(4)	stop only: starts at 0, step 1	[0, 1, 2, 3]
range(5, 10)	start and stop: step 1	[5, 6, 7, 8, 9]
range(1, 12, 4)	start, stop and step	[1, 5, 9]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(1, 11)**, which runs the body exactly 10 times.

Program

```
name = input("Enter your name: ")
for count in range(1, 11):
    print(count, ": Welcome to Python,", name)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter your name: Asha
1 : Welcome to Python, Asha
2 : Welcome to Python, Asha
3 : Welcome to Python, Asha
4 : Welcome to Python, Asha
5 : Welcome to Python, Asha
6 : Welcome to Python, Asha
7 : Welcome to Python, Asha
8 : Welcome to Python, Asha
9 : Welcome to Python, Asha
10 : Welcome to Python, Asha
```

Answer 13 - Set 3 of 10 (Squares of 1 to 10)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(6)	stop only: starts at 0, step 1	[0, 1, 2, 3, 4, 5]
range(2, 7)	start and stop: step 1	[2, 3, 4, 5, 6]
range(0, 20, 5)	start, stop and step	[0, 5, 10, 15]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(1, 11)**, which runs the body exactly 10 times.

Program

```
for k in range(1, 11):
    print("Square of", k, "is", k * k)
```

Sample Input / Output (typed values appear after each prompt)

```
Square of 1 is 1
Square of 2 is 4
Square of 3 is 9
Square of 4 is 16
Square of 5 is 25
Square of 6 is 36
Square of 7 is 49
Square of 8 is 64
Square of 9 is 81
Square of 10 is 100
```

Answer 14 - Set 4 of 10 (Countdown (negative step))

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(3)	stop only: starts at 0, step 1	[0, 1, 2]
range(4, 9)	start and stop: step 1	[4, 5, 6, 7, 8]
range(10, 0, -2)	start, stop and step	[10, 8, 6, 4, 2]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(10, 0, -1)**, which runs the body exactly 10 times.

Program

```
start = int(input("Start countdown from: "))
for n in range(start, 0, -1):
    print(n)
print("Lift off!")
```

Sample Input / Output (typed values appear after each prompt)

```
Start countdown from: 10
10
9
8
7
6
5
4
3
2
1
Lift off!
```

Answer 15 - Set 5 of 10 (First 10 even numbers)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(8)	stop only: starts at 0, step 1	[0, 1, 2, 3, 4, 5, 6, 7]
range(6, 12)	start and stop: step 1	[6, 7, 8, 9, 10, 11]
range(5, 30, 5)	start, stop and step	[5, 10, 15, 20, 25]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(2, 21, 2)**, which runs the body exactly 10 times.

Program

```
print("First 10 even numbers:")
for e in range(2, 21, 2):
    print(e, end=" ")
print()
```

Sample Input / Output (typed values appear after each prompt)

```
First 10 even numbers:
2 4 6 8 10 12 14 16 18 20
```

Answer 16 - Set 6 of 10 (Sum of 10 numbers entered by user)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(5)	stop only: starts at 0, step 1	[0, 1, 2, 3, 4]
range(1, 6)	start and stop: step 1	[1, 2, 3, 4, 5]
range(3, 18, 5)	start, stop and step	[3, 8, 13]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(10)**, which runs the body exactly 10 times.

Program

```
total = 0
for idx in range(10):
    value = int(input("Enter number " + str(idx + 1) + ": "))
    total = total + value
print("Sum of the 10 numbers =", total)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter number 1: 12
Enter number 2: 7
Enter number 3: 25
Enter number 4: 3
Enter number 5: 18
Enter number 6: 9
Enter number 7: 30
Enter number 8: 4
Enter number 9: 11
Enter number 10: 21
Sum of the 10 numbers = 140
```

Answer 17 - Set 7 of 10 (Cubes of 1 to 10)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(9)	stop only: starts at 0, step 1	[0, 1, 2, 3, 4, 5, 6, 7, 8]
range(3, 8)	start and stop: step 1	[3, 4, 5, 6, 7]
range(1, 10, 3)	start, stop and step	[1, 4, 7]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(1, 11)**, which runs the body exactly 10 times.

Program

```
for c in range(1, 11):
    print(f"{c} cubed = {c ** 3}")
```

Sample Input / Output (typed values appear after each prompt)

```
1 cubed = 1
2 cubed = 8
3 cubed = 27
4 cubed = 64
5 cubed = 125
6 cubed = 216
7 cubed = 343
8 cubed = 512
9 cubed = 729
```

Answer 18 - Set 8 of 10 (Star pattern of 10 rows)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(5)	stop only: starts at 0, step 1	[0, 1, 2, 3, 4]
range(2, 8)	start and stop: step 1	[2, 3, 4, 5, 6, 7]
range(1, 16, 5)	start, stop and step	[1, 6, 11]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(1, 11)**, which runs the body exactly 10 times.

Program

```
for row in range(1, 11):
    print("*" * row)
```

Sample Input / Output (typed values appear after each prompt)

```
*
**
***
****
*****
*****
*****
*****
*****
*****
*****
```

Answer 19 - Set 9 of 10 (First 10 odd numbers)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(7)	stop only: starts at 0, step 1	[0, 1, 2, 3, 4, 5, 6]
range(5, 11)	start and stop: step 1	[5, 6, 7, 8, 9, 10]
range(1, 25, 6)	start, stop and step	[1, 7, 13, 19]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(1, 20, 2)**, which runs the body exactly 10 times.

Program

```
for odd in range(1, 20, 2):
    print("Odd number:", odd)
```

Sample Input / Output (typed values appear after each prompt)

```
Odd number: 1
Odd number: 3
Odd number: 5
Odd number: 7
Odd number: 9
Odd number: 11
Odd number: 13
Odd number: 15
Odd number: 17
Odd number: 19
```

Answer 20 - Set 10 of 10 (Average of 10 marks)

The **for loop** repeats a block of statements once for every item of a sequence. Syntax: **for variable in sequence:** followed by an indented body. The **range()** function generates a sequence of integers: **range(stop)**, **range(start, stop)** or **range(start, stop, step)**. The stop value is never included. Examples of the three forms:

Forms of range()

Call	Meaning	Generated values
range(6)	stop only: starts at 0, step 1	[0, 1, 2, 3, 4, 5]
range(4, 10)	start and stop: step 1	[4, 5, 6, 7, 8, 9]
range(0, 30, 7)	start, stop and step	[0, 7, 14, 21, 28]

Execute statements exactly 10 times: a **for loop with range()** is used. In the program below it is written as **for ... in range(10)**, which runs the body exactly 10 times.

Program

```
sum_marks = 0
for subject in range(10):
    m = float(input("Enter marks of subject " + str(subject + 1) + ": "))
    sum_marks += m
average = sum_marks / 10
print("Total marks =", sum_marks)
print("Average marks =", average)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter marks of subject 1: 78
Enter marks of subject 2: 65
Enter marks of subject 3: 90
Enter marks of subject 4: 82
Enter marks of subject 5: 71
Enter marks of subject 6: 88
Enter marks of subject 7: 95
Enter marks of subject 8: 60
Enter marks of subject 9: 74
Enter marks of subject 10: 83
Total marks = 786.0
Average marks = 78.6
```

Section 3 : Group 1 (Roll Number ends in 0, 3, 5, 7) - Question 3 (Answers 21 - 30)

Question: Write a Python program to accept the name, roll no, age and percentage of a student and display the values along with their data types. Write the algorithm and flowchart for the same.

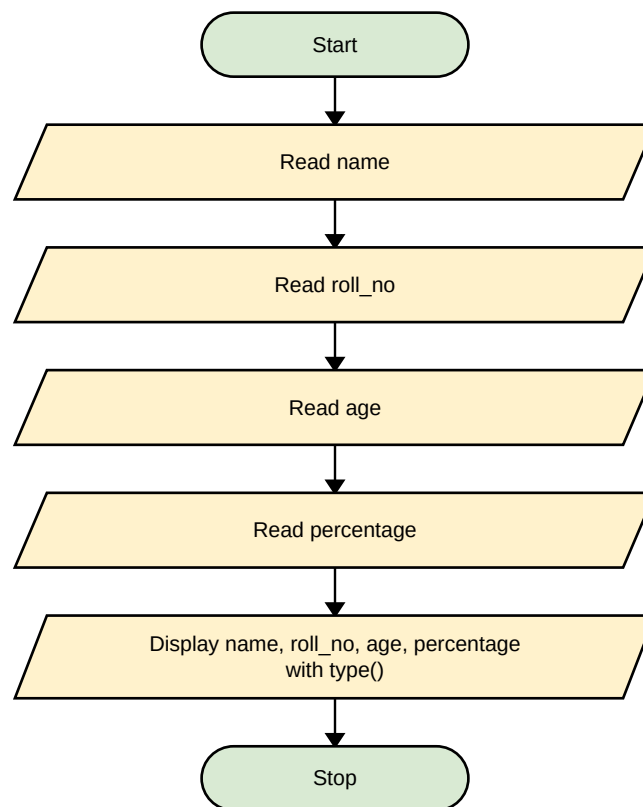
Answer 21 - Set 1 of 10 (Student: Aarav Mehta)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **name** (string).
3. Read the roll number into **roll_no** (integer).
4. Read the age into **age** (integer).
5. Read the percentage into **percentage** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
name = input("Enter name: ")
roll_no = int(input("Enter roll no: "))
age = int(input("Enter age: "))
percentage = float(input("Enter percentage: "))
print("--- Student Details ---")
print("Name :", name, type(name))
print("Roll No :", roll_no, type(roll_no))
print("Age :", age, type(age))
print("Percentage :", percentage, type(percentage))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter name: Aarav Mehta
Enter roll no: 101
Enter age: 18
Enter percentage: 87.5
--- Student Details ---
Name : Aarav Mehta <class 'str'>
Roll No : 101 <class 'int'>
Age : 18 <class 'int'>
Percentage : 87.5 <class 'float'>
```

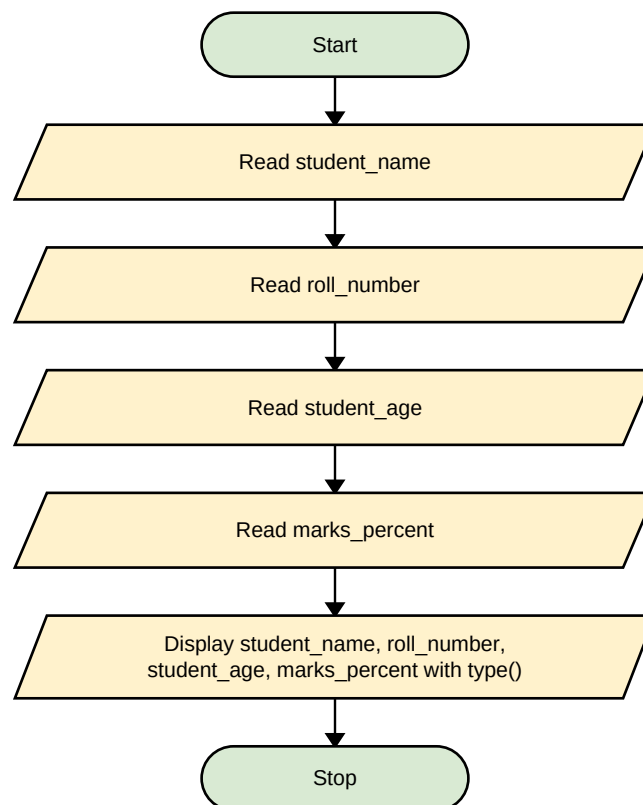
Answer 22 - Set 2 of 10 (Student: Diya Shetty)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **student_name** (string).
3. Read the roll number into **roll_number** (integer).
4. Read the age into **student_age** (integer).
5. Read the percentage into **marks_percent** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
student_name = input("Name: ")
roll_number = int(input("Roll No: "))
student_age = int(input("Age: "))
marks_percent = float(input("Percentage: "))
print("--- Student Details ---")
print(f"Name = {student_name}    Type = {type(student_name)}")
print(f"Roll No = {roll_number}    Type = {type(roll_number)}")
print(f"Age = {student_age}    Type = {type(student_age)}")
print(f"Percentage = {marks_percent}    Type = {type(marks_percent)}")
```

Sample Input / Output (typed values appear after each prompt)

```
Name: Diya Shetty
Roll No: 214
Age: 19
Percentage: 92.4
--- Student Details ---
Name = Diya Shetty   Type = <class 'str'>
Roll No = 214       Type = <class 'int'>
Age = 19           Type = <class 'int'>
Percentage = 92.4   Type = <class 'float'>
```

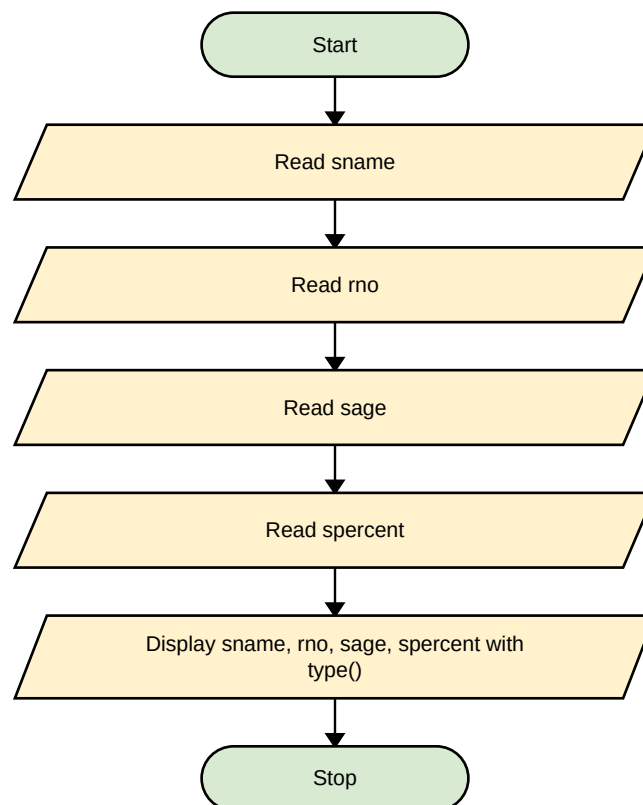
Answer 23 - Set 3 of 10 (Student: Kiran Gowda)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **sname** (string).
3. Read the roll number into **rno** (integer).
4. Read the age into **sage** (integer).
5. Read the percentage into **spercent** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
sname = input("Please enter the student's name: ")
rno = int(input("Please enter the roll number: "))
sage = int(input("Please enter the age: "))
spercent = float(input("Please enter the percentage: "))
print("--- Student Details ---")
print("Name:", sname, "| data type:", type(sname).__name__)
print("Roll No:", rno, "| data type:", type(rno).__name__)
print("Age:", sage, "| data type:", type(sage).__name__)
print("Percentage:", sppercent, "| data type:", type(spercent).__name__)
```

Sample Input / Output (typed values appear after each prompt)

```
Please enter the student's name: Kiran Gowda
Please enter the roll number: 307
Please enter the age: 17
Please enter the percentage: 76.8
--- Student Details ---
Name: Kiran Gowda | data type: str
Roll No: 307 | data type: int
Age: 17 | data type: int
Percentage: 76.8 | data type: float
```

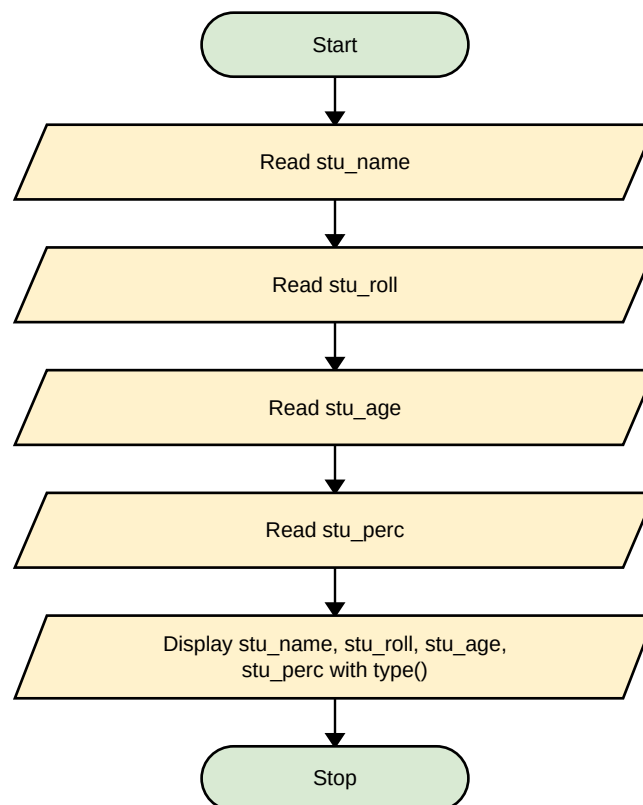
Answer 24 - Set 4 of 10 (Student: Sneha Bhat)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **stu_name** (string).
3. Read the roll number into **stu_roll** (integer).
4. Read the age into **stu_age** (integer).
5. Read the percentage into **stu_perc** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
stu_name = input("Enter name: ")
stu_roll = int(input("Enter roll no: "))
stu_age = int(input("Enter age: "))
stu_perc = float(input("Enter percentage: "))
print("--- Student Details ---")
print("Name :", stu_name, type(stu_name))
print("Roll No :", stu_roll, type(stu_roll))
print("Age :", stu_age, type(stu_age))
print("Percentage :", stu_perc, type(stu_perc))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter name: Sneha Bhat
Enter roll no: 118
Enter age: 18
Enter percentage: 81.25
--- Student Details ---
Name : Sneha Bhat <class 'str'>
Roll No : 118 <class 'int'>
Age : 18 <class 'int'>
Percentage : 81.25 <class 'float'>
```

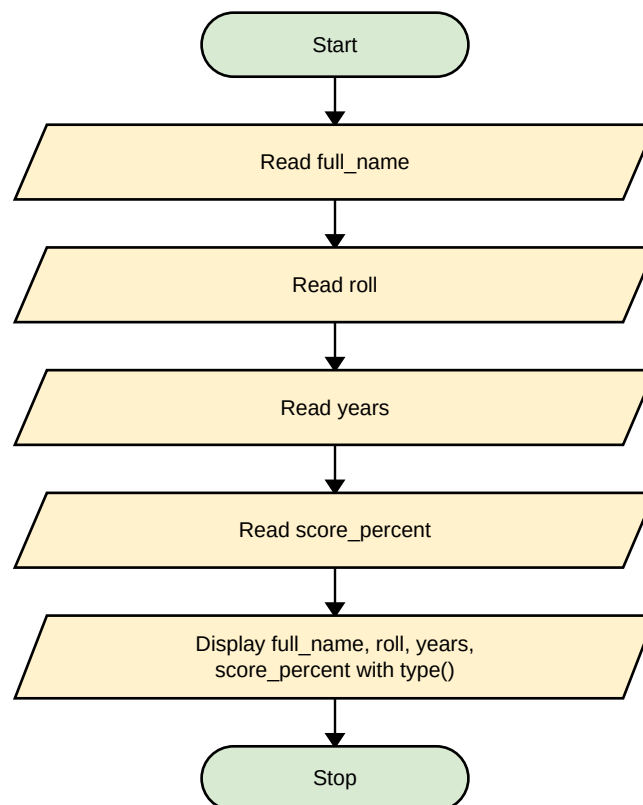
Answer 25 - Set 5 of 10 (Student: Rahul Verma)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **full_name** (string).
3. Read the roll number into **roll** (integer).
4. Read the age into **years** (integer).
5. Read the percentage into **score_percent** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
full_name = input("Name: ")
roll = int(input("Roll No: "))
years = int(input("Age: "))
score_percent = float(input("Percentage: "))
print("--- Student Details ---")
print(f"Name = {full_name}    Type = {type(full_name)}")
print(f"Roll No = {roll}    Type = {type(roll)}")
print(f"Age = {years}    Type = {type(years)}")
print(f"Percentage = {score_percent}    Type = {type(score_percent)}")
```

Sample Input / Output (typed values appear after each prompt)

```
Name: Rahul Verma
Roll No: 425
Age: 20
Percentage: 69.5
--- Student Details ---
Name = Rahul Verma    Type = <class 'str'>
Roll No = 425         Type = <class 'int'>
Age = 20              Type = <class 'int'>
Percentage = 69.5     Type = <class 'float'>
```

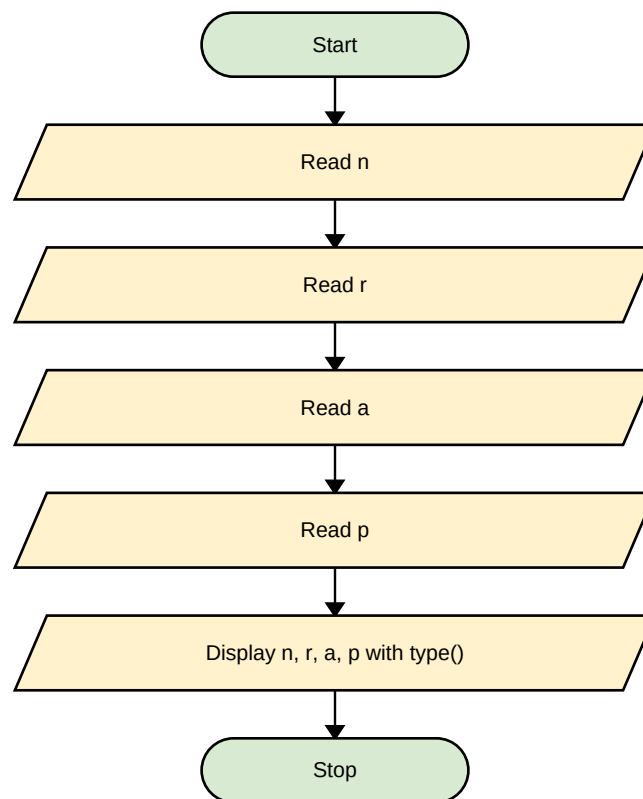
Answer 26 - Set 6 of 10 (Student: Nisha Pillai)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **n** (string).
3. Read the roll number into **r** (integer).
4. Read the age into **a** (integer).
5. Read the percentage into **p** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
n = input("Please enter the student's name: ")
r = int(input("Please enter the roll number: "))
a = int(input("Please enter the age: "))
p = float(input("Please enter the percentage: "))
print("--- Student Details ---")
print("Name:", n, "| data type:", type(n).__name__)
print("Roll No:", r, "| data type:", type(r).__name__)
print("Age:", a, "| data type:", type(a).__name__)
print("Percentage:", p, "| data type:", type(p).__name__)
```

Sample Input / Output (typed values appear after each prompt)

```
Please enter the student's name: Nisha Pillai
Please enter the roll number: 232
Please enter the age: 18
Please enter the percentage: 94.0
--- Student Details ---
Name: Nisha Pillai | data type: str
Roll No: 232 | data type: int
Age: 18 | data type: int
Percentage: 94.0 | data type: float
```

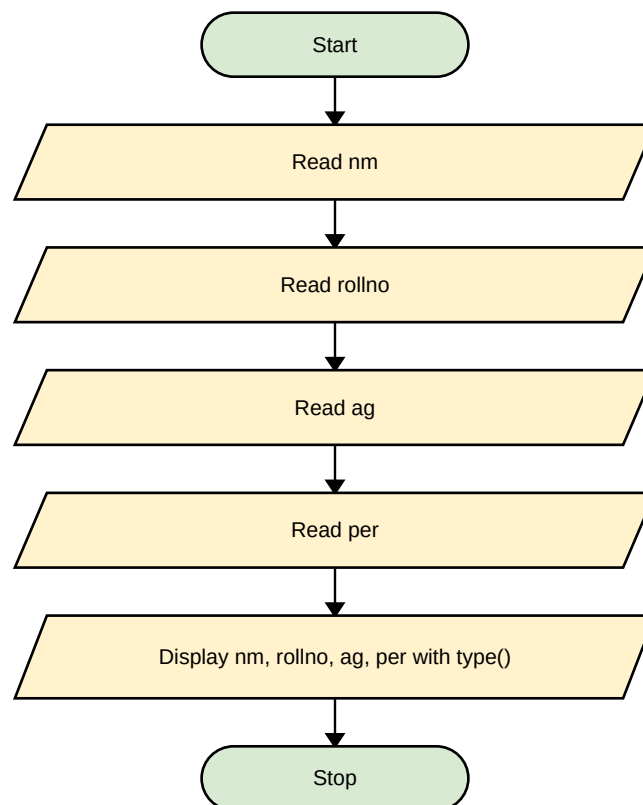
Answer 27 - Set 7 of 10 (Student: Varun Hegde)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **nm** (string).
3. Read the roll number into **rollno** (integer).
4. Read the age into **ag** (integer).
5. Read the percentage into **per** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
nm = input("Enter name: ")
rollno = int(input("Enter roll no: "))
ag = int(input("Enter age: "))
per = float(input("Enter percentage: "))
print("--- Student Details ---")
print("Name :", nm, type(nm))
print("Roll No :", rollno, type(rollno))
print("Age :", ag, type(ag))
print("Percentage :", per, type(per))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter name: Varun Hegde
Enter roll no: 156
Enter age: 19
Enter percentage: 72.6
--- Student Details ---
Name : Varun Hegde <class 'str'>
Roll No : 156 <class 'int'>
Age : 19 <class 'int'>
Percentage : 72.6 <class 'float'>
```

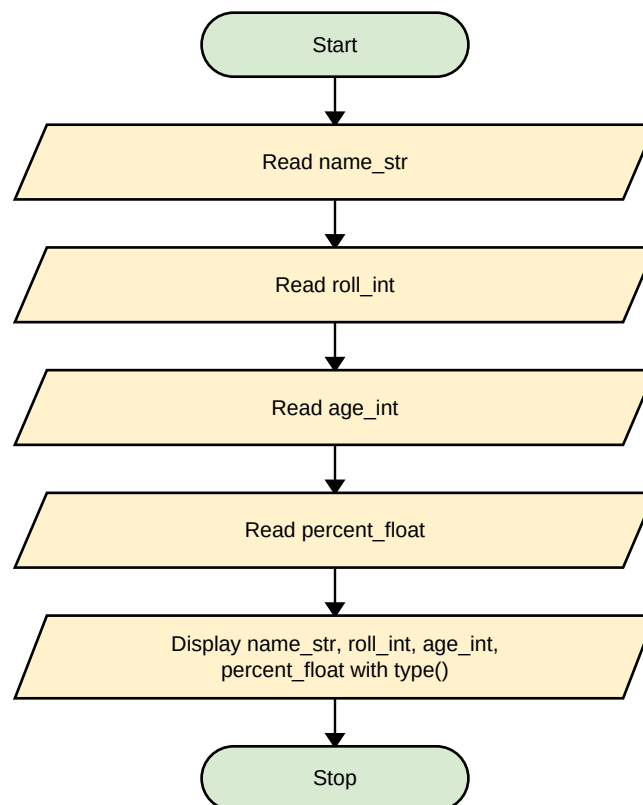
Answer 28 - Set 8 of 10 (Student: Ishita Das)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **name_str** (string).
3. Read the roll number into **roll_int** (integer).
4. Read the age into **age_int** (integer).
5. Read the percentage into **percent_float** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
name_str = input("Name: ")
roll_int = int(input("Roll No: "))
age_int = int(input("Age: "))
percent_float = float(input("Percentage: "))
print("--- Student Details ---")
print(f"Name = {name_str}    Type = {type(name_str)}")
print(f"Roll No = {roll_int}    Type = {type(roll_int)}")
print(f"Age = {age_int}    Type = {type(age_int)}")
print(f"Percentage = {percent_float}    Type = {type(percent_float)}")
```

Sample Input / Output (typed values appear after each prompt)

```
Name: Ishita Das
Roll No: 349
Age: 18
Percentage: 88.9
--- Student Details ---
Name = Ishita Das   Type = <class 'str'>
Roll No = 349      Type = <class 'int'>
Age = 18           Type = <class 'int'>
Percentage = 88.9   Type = <class 'float'>
```

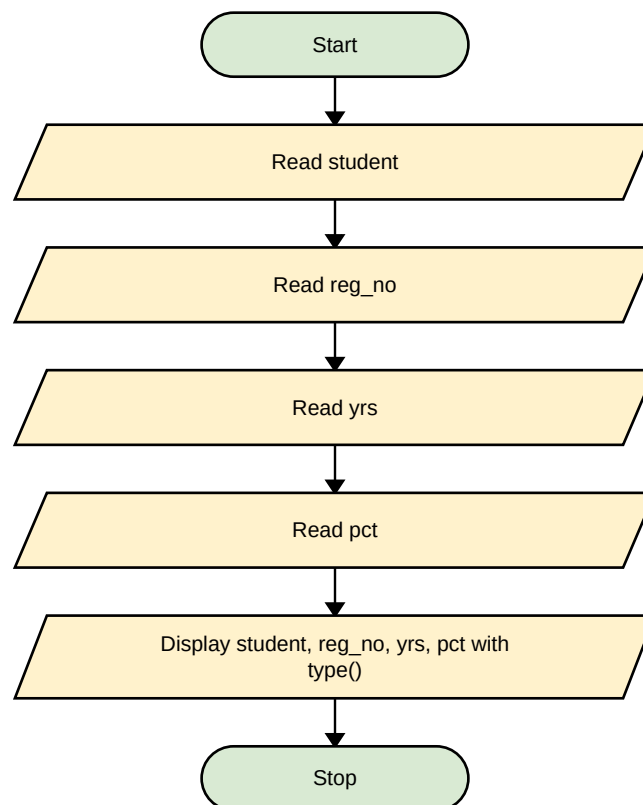
Answer 29 - Set 9 of 10 (Student: Tanmay Rao)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **student** (string).
3. Read the roll number into **reg_no** (integer).
4. Read the age into **yrs** (integer).
5. Read the percentage into **pct** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
student = input("Please enter the student's name: ")
reg_no = int(input("Please enter the roll number: "))
yrs = int(input("Please enter the age: "))
pct = float(input("Please enter the percentage: "))
print("--- Student Details ---")
print("Name:", student, "| data type:", type(student).__name__)
print("Roll No:", reg_no, "| data type:", type(reg_no).__name__)
print("Age:", yrs, "| data type:", type(yrs).__name__)
print("Percentage:", pct, "| data type:", type(pct).__name__)
```

Sample Input / Output (typed values appear after each prompt)

```
Please enter the student's name: Tanmay Rao
Please enter the roll number: 271
Please enter the age: 17
Please enter the percentage: 79.3
--- Student Details ---
Name: Tanmay Rao | data type: str
Roll No: 271 | data type: int
Age: 17 | data type: int
Percentage: 79.3 | data type: float
```

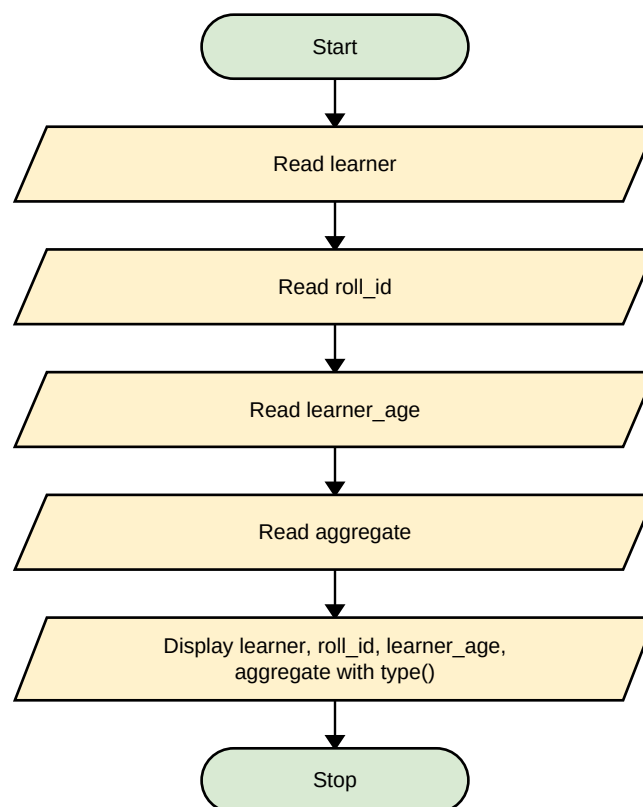
Answer 30 - Set 10 of 10 (Student: Lakshmi Menon)

The **input()** function always returns a string, so the roll number and age are converted with **int()** and the percentage with **float()**. The **type()** function returns the data type of a value, and **print()** displays the values.

Algorithm

1. Start.
2. Read the student's name into **learner** (string).
3. Read the roll number into **roll_id** (integer).
4. Read the age into **learner_age** (integer).
5. Read the percentage into **aggregate** (float).
6. Display the value of each variable together with its data type using **type()**.
7. Stop.

Flowchart



Program

```
learner = input("Enter name: ")
roll_id = int(input("Enter roll no: "))
learner_age = int(input("Enter age: "))
aggregate = float(input("Enter percentage: "))
print("--- Student Details ---")
print("Name :", learner, type(learner))
print("Roll No :", roll_id, type(roll_id))
print("Age :", learner_age, type(learner_age))
print("Percentage :", aggregate, type(aggregate))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter name: Lakshmi Menon
Enter roll no: 483
Enter age: 19
Enter percentage: 90.75
--- Student Details ---
Name : Lakshmi Menon <class 'str'>
Roll No : 483 <class 'int'>
Age : 19 <class 'int'>
Percentage : 90.75 <class 'float'>
```

Section 4 : Group 2 (Roll Number ends in 1, 6, 8) - Question 1 (Answers 31 - 40)

Question: Explain Bitwise XOR, left shift (<<) and right shift (>>) operators with examples. A number-processing application executes $8 \ll 2$. Explain how the left-shift and right shift operation changes the binary representation and determine the result.

Answer 31 - Set 1 of 10 (XOR of 12 and 10, shift of 20 by 2)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 12, 10 and 20.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
$\wedge$ (XOR)	Result bit is 1 only when the two bits differ	$12 \wedge 10$	$00001100 \wedge 00001010 = 00000110 = 6$
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	$20 \ll 2$	$00010100 \rightarrow 0001010000 = 80$
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	$20 \gg 2$	$00010100 \rightarrow 00000101 = 5$

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("a =", format(a, "08b"), " b =", format(b, "08b"))
print("a ^ b =", a ^ b, "-> binary", format(a ^ b, "08b"))
n = int(input("Enter number to shift: "))
k = int(input("Enter shift count: "))
print("n << k =", n << k, "-> binary", format(n << k, "b"))
print("n >> k =", n >> k, "-> binary", format(n >> k, "b"))
# Number-processing application: 8 << 2
number = 8
shift_by = 2
print("Binary of", number, "=", format(number, "08b"))
left = number << shift_by
print(number, "<<", shift_by, "=", left, "-> binary", format(left, "08b"))
right = number >> shift_by
print(number, ">>", shift_by, "=", right, "-> binary", format(right, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 12
Enter second number: 10
a = 00001100   b = 00001010
a ^ b = 6 -> binary 00000110
Enter number to shift: 20
Enter shift count: 2
n << k = 80 -> binary 1010000
n >> k = 5 -> binary 101
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 32 - Set 2 of 10 (XOR of 25 and 7, shift of 13 by 3)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 25, 7 and 13.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
^ (XOR)	Result bit is 1 only when the two bits differ	25 ^ 7	00011001 ^ 00000111 = 00011110 = 30
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	13 << 3	00001101 -> 0001101000 = 104
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	13 >> 3	00001101 -> 00000001 = 1

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))
print("x =", format(x, "08b"), " y =", format(y, "08b"))
print("x ^ y =", x ^ y, "-> binary", format(x ^ y, "08b"))
val = int(input("Enter number to shift: "))
sh = int(input("Enter shift count: "))
print("val << sh =", val << sh, "-> binary", format(val << sh, "b"))
print("val >> sh =", val >> sh, "-> binary", format(val >> sh, "b"))
# Number-processing application:  $8 \ll 2$ 
num = 8
places = 2
print("Binary of", num, "=", format(num, "08b"))
l_res = num << places
print(num, "<<", places, "=", l_res, "-> binary", format(l_res, "08b"))
r_res = num >> places
print(num, ">>", places, "=", r_res, "-> binary", format(r_res, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 25
Enter second number: 7
x = 00011001 y = 00000111
x ^ y = 30 -> binary 00011110
Enter number to shift: 13
Enter shift count: 3
val << sh = 104 -> binary 1101000
val >> sh = 1 -> binary 1
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 33 - Set 3 of 10 (XOR of 9 and 5, shift of 24 by 2)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 9, 5 and 24.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
^ (XOR)	Result bit is 1 only when the two bits differ	9 ^ 5	00001001 ^ 00000101 = 00001100 = 12
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	24 << 2	00011000 -> 0001100000 = 96
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	24 >> 2	00011000 -> 00000110 = 6

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
p = int(input("Enter first number: "))
q = int(input("Enter second number: "))
print("p =", format(p, "08b"), " q =", format(q, "08b"))
print("p ^ q =", p ^ q, "-> binary", format(p ^ q, "08b"))
m = int(input("Enter number to shift: "))
cnt = int(input("Enter shift count: "))
print("m << cnt =", m << cnt, "-> binary", format(m << cnt, "b"))
print("m >> cnt =", m >> cnt, "-> binary", format(m >> cnt, "b"))
# Number-processing application: 8 << 2
value = 8
amount = 2
print("Binary of", value, "=", format(value, "08b"))
shifted_left = value << amount
print(value, "<<", amount, "=", shifted_left, "-> binary", format(shifted_left, "08b"))
shifted_right = value >> amount
print(value, ">>", amount, "=", shifted_right, "-> binary", format(shifted_right, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 9
Enter second number: 5
p = 00001001   q = 00000101
p ^ q = 12 -> binary 00001100
Enter number to shift: 24
Enter shift count: 2
m << cnt = 96 -> binary 1100000
m >> cnt = 6 -> binary 110
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 34 - Set 4 of 10 (XOR of 40 and 18, shift of 50 by 1)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 40, 18 and 50.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
^ (XOR)	Result bit is 1 only when the two bits differ	40 ^ 18	00101000 ^ 00010010 = 00111010 = 58
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	50 << 1	00110010 -> 0001100100 = 100
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	50 >> 1	00110010 -> 00011001 = 25

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
n1 = int(input("Enter first number: "))
n2 = int(input("Enter second number: "))
print("n1 =", format(n1, "08b"), " n2 =", format(n2, "08b"))
print("n1 ^ n2 =", n1 ^ n2, "-> binary", format(n1 ^ n2, "08b"))
base = int(input("Enter number to shift: "))
steps = int(input("Enter shift count: "))
print("base << steps =", base << steps, "-> binary", format(base << steps, "b"))
print("base >> steps =", base >> steps, "-> binary", format(base >> steps, "b"))
# Number-processing application: 8 << 2
data = 8
move = 2
print("Binary of", data, "=", format(data, "08b"))
ls = data << move
print(data, "<<", move, "=", ls, "-> binary", format(ls, "08b"))
rs = data >> move
print(data, ">>", move, "=", rs, "-> binary", format(rs, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 40
Enter second number: 18
n1 = 00101000   n2 = 00010010
n1 ^ n2 = 58 -> binary 00111010
Enter number to shift: 50
Enter shift count: 1
base << steps = 100 -> binary 1100100
base >> steps = 25 -> binary 11001
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 35 - Set 5 of 10 (XOR of 15 and 6, shift of 36 by 3)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 15, 6 and 36.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
^ (XOR)	Result bit is 1 only when the two bits differ	15 ^ 6	00001111 ^ 00000110 = 00001001 = 9
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	36 << 3	00100100 -> 0100100000 = 288
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	36 >> 3	00100100 -> 00000100 = 4

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
first = int(input("Enter first number: "))
second = int(input("Enter second number: "))
print("first =", format(first, "08b"), " second =", format(second, "08b"))
print("first ^ second =", first ^ second, "-> binary", format(first ^ second, "08b"))
item = int(input("Enter number to shift: "))
bits = int(input("Enter shift count: "))
print("item << bits =", item << bits, "-> binary", format(item << bits, "b"))
print("item >> bits =", item >> bits, "-> binary", format(item >> bits, "b"))
# Number-processing application: 8 << 2
digit = 8
count = 2
print("Binary of", digit, "=", format(digit, "08b"))
left_out = digit << count
print(digit, "<<", count, "=", left_out, "-> binary", format(left_out, "08b"))
right_out = digit >> count
print(digit, ">>", count, "=", right_out, "-> binary", format(right_out, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 15
Enter second number: 6
first = 00001111   second = 00000110
first ^ second = 9 -> binary 00001001
Enter number to shift: 36
Enter shift count: 3
item << bits = 288 -> binary 100100000
item >> bits = 4 -> binary 100
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 36 - Set 6 of 10 (XOR of 33 and 21, shift of 45 by 2)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 33, 21 and 45.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
^ (XOR)	Result bit is 1 only when the two bits differ	33 ^ 21	00100001 ^ 00010101 = 00110100 = 52
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	45 << 2	00101101 -> 0010110100 = 180
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	45 >> 2	00101101 -> 00001011 = 11

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
u = int(input("Enter first number: "))
v = int(input("Enter second number: "))
print("u =", format(u, "08b"), " v =", format(v, "08b"))
print("u ^ v =", u ^ v, "-> binary", format(u ^ v, "08b"))
w = int(input("Enter number to shift: "))
t = int(input("Enter shift count: "))
print("w << t =", w << t, "-> binary", format(w << t, "b"))
print("w >> t =", w >> t, "-> binary", format(w >> t, "b"))
# Number-processing application: 8 << 2
number = 8
n_bits = 2
print("Binary of", number, "=", format(number, "08b"))
result_l = number << n_bits
print(number, "<<", n_bits, "=", result_l, "-> binary", format(result_l, "08b"))
result_r = number >> n_bits
print(number, ">>", n_bits, "=", result_r, "-> binary", format(result_r, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 33
Enter second number: 21
u = 00100001 v = 00010101
u ^ v = 52 -> binary 00110100
Enter number to shift: 45
Enter shift count: 2
w << t = 180 -> binary 10110100
w >> t = 11 -> binary 1011
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 37 - Set 7 of 10 (XOR of 60 and 13, shift of 72 by 3)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 60, 13 and 72.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
^ (XOR)	Result bit is 1 only when the two bits differ	60 ^ 13	00111100 ^ 00001101 = 00110001 = 49
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	72 << 3	01001000 -> 1001000000 = 576
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	72 >> 3	01001000 -> 00001001 = 9

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print("num1 =", format(num1, "08b"), " num2 =", format(num2, "08b"))
print("num1 ^ num2 =", num1 ^ num2, "-> binary", format(num1 ^ num2, "08b"))
target = int(input("Enter number to shift: "))
by = int(input("Enter shift count: "))
print("target << by =", target << by, "-> binary", format(target << by, "b"))
print("target >> by =", target >> by, "-> binary", format(target >> by, "b"))
# Number-processing application: 8 << 2
x0 = 8
s0 = 2
print("Binary of", x0, "=", format(x0, "08b"))
lres = x0 << s0
print(x0, "<<", s0, "=", lres, "-> binary", format(lres, "08b"))
rres = x0 >> s0
print(x0, ">>", s0, "=", rres, "-> binary", format(rres, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 60
Enter second number: 13
num1 = 00111100 num2 = 00001101
num1 ^ num2 = 49 -> binary 00110001
Enter number to shift: 72
Enter shift count: 3
target << by = 576 -> binary 1001000000
target >> by = 9 -> binary 1001
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 38 - Set 8 of 10 (XOR of 29 and 14, shift of 19 by 4)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 29, 14 and 19.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
^ (XOR)	Result bit is 1 only when the two bits differ	29 ^ 14	00011101 ^ 00001110 = 00010011 = 19
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	19 << 4	00010011 -> 0100110000 = 304
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	19 >> 4	00010011 -> 00000001 = 1

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
s = int(input("Enter first number: "))
t = int(input("Enter second number: "))
print("s =", format(s, "08b"), " t =", format(t, "08b"))
print("s ^ t =", s ^ t, "-> binary", format(s ^ t, "08b"))
z = int(input("Enter number to shift: "))
j = int(input("Enter shift count: "))
print("z << j =", z << j, "-> binary", format(z << j, "b"))
print("z >> j =", z >> j, "-> binary", format(z >> j, "b"))
# Number-processing application: 8 << 2
orig = 8
dist = 2
print("Binary of", orig, "=", format(orig, "08b"))
after_left = orig << dist
print(orig, "<<", dist, "=", after_left, "-> binary", format(after_left, "08b"))
after_right = orig >> dist
print(orig, ">>", dist, "=", after_right, "-> binary", format(after_right, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 29
Enter second number: 14
s = 00011101    t = 00001110
s ^ t = 19 -> binary 00010011
Enter number to shift: 19
Enter shift count: 4
z << j = 304 -> binary 100110000
z >> j = 1 -> binary 1
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 39 - Set 9 of 10 (XOR of 48 and 27, shift of 88 by 2)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 48, 27 and 88.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
^ (XOR)	Result bit is 1 only when the two bits differ	48 ^ 27	00110000 ^ 00011011 = 00101011 = 43
<< (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	88 << 2	01011000 -> 0101100000 = 352
>> (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	88 >> 2	01011000 -> 00010110 = 22

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
val1 = int(input("Enter first number: "))
val2 = int(input("Enter second number: "))
print("val1 =", format(val1, "08b"), " val2 =", format(val2, "08b"))
print("val1 ^ val2 =", val1 ^ val2, "-> binary", format(val1 ^ val2, "08b"))
inp = int(input("Enter number to shift: "))
pos = int(input("Enter shift count: "))
print("inp << pos =", inp << pos, "-> binary", format(inp << pos, "b"))
print("inp >> pos =", inp >> pos, "-> binary", format(inp >> pos, "b"))
# Number-processing application: 8 << 2
eight = 8
two = 2
print("Binary of", eight, "=", format(eight, "08b"))
ans_left = eight << two
print(eight, "<<", two, "=", ans_left, "-> binary", format(ans_left, "08b"))
ans_right = eight >> two
print(eight, ">>", two, "=", ans_right, "-> binary", format(ans_right, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 48
Enter second number: 27
val1 = 00110000    val2 = 00011011
val1 ^ val2 = 43 -> binary 00101011
Enter number to shift: 88
Enter shift count: 2
inp << pos = 352 -> binary 101100000
inp >> pos = 22 -> binary 10110
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Answer 40 - Set 10 of 10 (XOR of 37 and 11, shift of 100 by 3)

Bitwise operators work on the binary representation of integers, bit by bit. The examples below use 37, 11 and 100.

Bitwise XOR, left shift and right shift

Operator	Description	Example	Working
$\wedge$ (XOR)	Result bit is 1 only when the two bits differ	$37 \wedge 11$	$00100101 \wedge 00001011 = 00101110 = 46$
$\ll$ (left shift)	Shifts bits left, zeros enter on the right: $x \ll n = x * 2^n$	$100 \ll 3$	$01100100 \rightarrow 110010000 = 800$
$\gg$ (right shift)	Shifts bits right, low bits are dropped: $x \gg n = x // 2^n$	$100 \gg 3$	$01100100 \rightarrow 00001100 = 12$

Application: $8 \ll 2$. 8 in binary is 00001000. Left shift by 2 moves every bit two places left and fills the right with zeros, giving 00100000, which is **32** ($8 \times 2^2 = 32$). So the binary pattern grows (its value is multiplied by 4). The right shift $8 \gg 2$ moves the bits two places right, giving 00000010 = 2 ($8 // 4$), so the value is divided by 4. **Result of $8 \ll 2 = 32$.**

Program

```
m1 = int(input("Enter first number: "))
m2 = int(input("Enter second number: "))
print("m1 =", format(m1, "08b"), " m2 =", format(m2, "08b"))
print("m1 ^ m2 =", m1 ^ m2, "-> binary", format(m1 ^ m2, "08b"))
q1 = int(input("Enter number to shift: "))
q2 = int(input("Enter shift count: "))
print("q1 << q2 =", q1 << q2, "-> binary", format(q1 << q2, "b"))
print("q1 >> q2 =", q1 >> q2, "-> binary", format(q1 >> q2, "b"))
# Number-processing application: 8 << 2
number = 8
shift_by = 2
print("Binary of", number, "=", format(number, "08b"))
out_left = number << shift_by
print(number, "<<", shift_by, "=", out_left, "-> binary", format(out_left, "08b"))
out_right = number >> shift_by
print(number, ">>", shift_by, "=", out_right, "-> binary", format(out_right, "08b"))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 37
Enter second number: 11
m1 = 00100101 m2 = 00001011
m1 ^ m2 = 46 -> binary 00101110
Enter number to shift: 100
Enter shift count: 3
q1 << q2 = 800 -> binary 1100100000
q1 >> q2 = 12 -> binary 1100
Binary of 8 = 00001000
8 << 2 = 32 -> binary 00100000
8 >> 2 = 2 -> binary 00000010
```

Section 5 : Group 2 (Roll Number ends in 1, 6, 8) - Question 2 (Answers 41 - 50)

Question: Write a Python program for a temperature-monitoring system that classifies temperature as Cold, Moderate, or Hot. Explain how if-elif-else can be used to implement this requirement. Write the algorithm and flowchart for the same.

Answer 41 - Set 1 of 10 (temp thresholds 15 / 30 (Celsius))

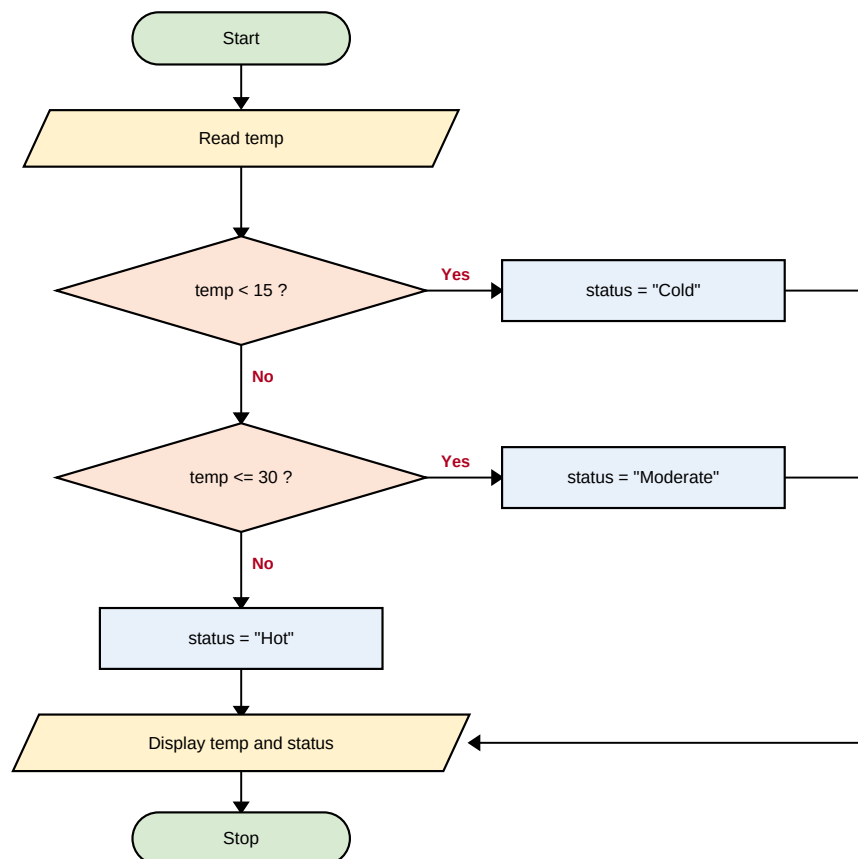
if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are:

Cold if temp < 15, **Moderate** if 15 <= temp <= 30, and **Hot** if temp > 30. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **temp**.
3. If **temp < 15**, set status to **Cold** and go to step 6.
4. Otherwise, if **temp <= 30**, set status to **Moderate** and go to step 6.
5. Otherwise (temp > 30), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```
temp = int(input("Enter the temperature in degrees Celsius: "))
if temp < 15:
    status = "Cold"
elif temp <= 30:
    status = "Moderate"
else:
    status = "Hot"
print("Temperature :", temp, "deg C")
```

```
print("Status      :", status)
```

Run 1 : sample input / output

```
Enter the temperature in degrees Celsius: 10
Temperature : 10 deg C
Status      : Cold
```

Run 2 : sample input / output

```
Enter the temperature in degrees Celsius: 22
Temperature : 22 deg C
Status      : Moderate
```

Run 3 : sample input / output

```
Enter the temperature in degrees Celsius: 36
Temperature : 36 deg C
Status      : Hot
```

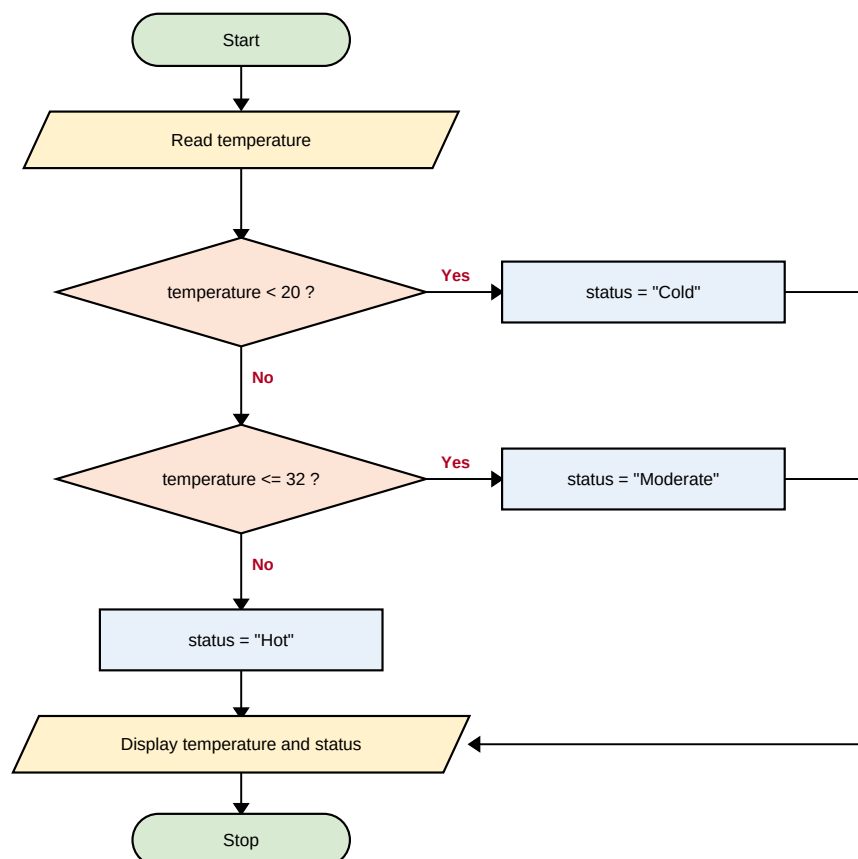
Answer 42 - Set 2 of 10 (temperature thresholds 20 / 32 (Celsius))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if temperature < 20, **Moderate** if 20 <= temperature <= 32, and **Hot** if temperature > 32. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **temperature**.
3. If **temperature < 20**, set status to **Cold** and go to step 6.
4. Otherwise, if **temperature <= 32**, set status to **Moderate** and go to step 6.
5. Otherwise (temperature > 32), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```
temperature = float(input("Enter the temperature in degrees Celsius: "))
if temperature < 20:
    status = "Cold"
elif temperature <= 32:
    status = "Moderate"
else:
    status = "Hot"
print(f"The temperature {temperature} deg C is classified as {status}.")
```

Run 1 : sample input / output

```
Enter the temperature in degrees Celsius: 14.5
The temperature 14.5 deg C is classified as Cold.
```

Run 2 : sample input / output

```
Enter the temperature in degrees Celsius: 26.5
The temperature 26.5 deg C is classified as Moderate.
```

Run 3 : sample input / output

```
Enter the temperature in degrees Celsius: 38.5
The temperature 38.5 deg C is classified as Hot.
```

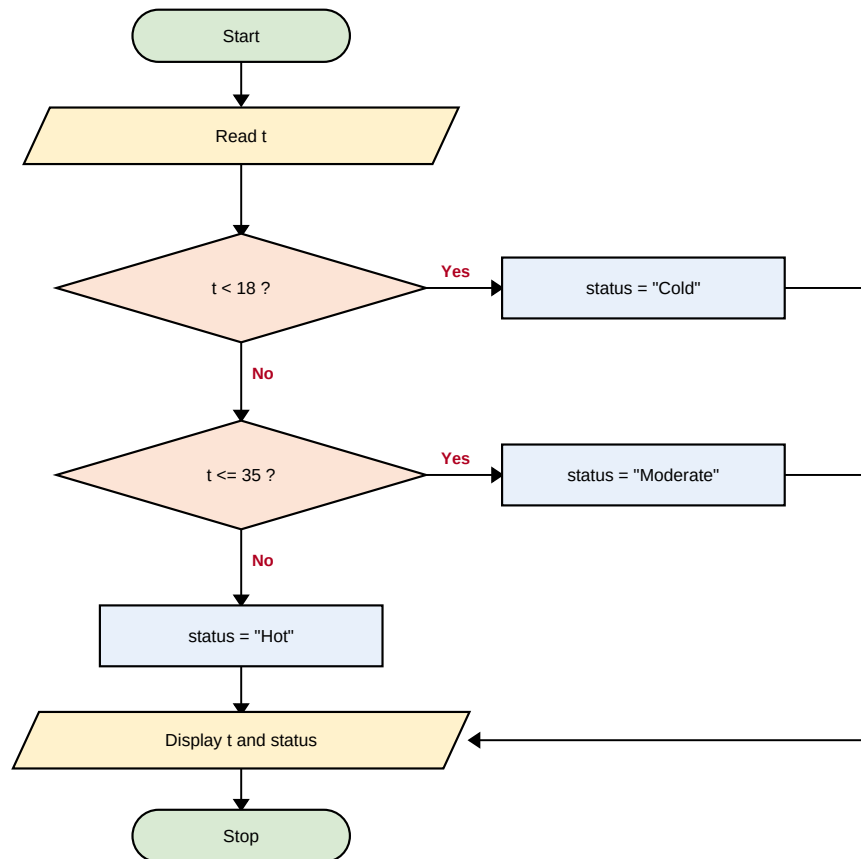
Answer 43 - Set 3 of 10 (t thresholds 18 / 35 (Celsius))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if $t < 18$, **Moderate** if $18 \leq t \leq 35$, and **Hot** if $t > 35$. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **t**.
3. If **t < 18**, set status to **Cold** and go to step 6.
4. Otherwise, if **t <= 35**, set status to **Moderate** and go to step 6.
5. Otherwise ($t > 35$), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```

t = int(input("Enter the temperature in degrees Celsius: "))
if t < 18:
    status = "Cold"
elif t <= 35:
    status = "Moderate"
else:
    status = "Hot"
print("Weather alert -> " + status.upper() + " (" + str(t) + " C)")

```

Run 1 : sample input / output

```

Enter the temperature in degrees Celsius: 13
Weather alert -> COLD (13 C)

```

Run 2 : sample input / output

```

Enter the temperature in degrees Celsius: 26
Weather alert -> MODERATE (26 C)

```

Run 3 : sample input / output

```

Enter the temperature in degrees Celsius: 41
Weather alert -> HOT (41 C)

```

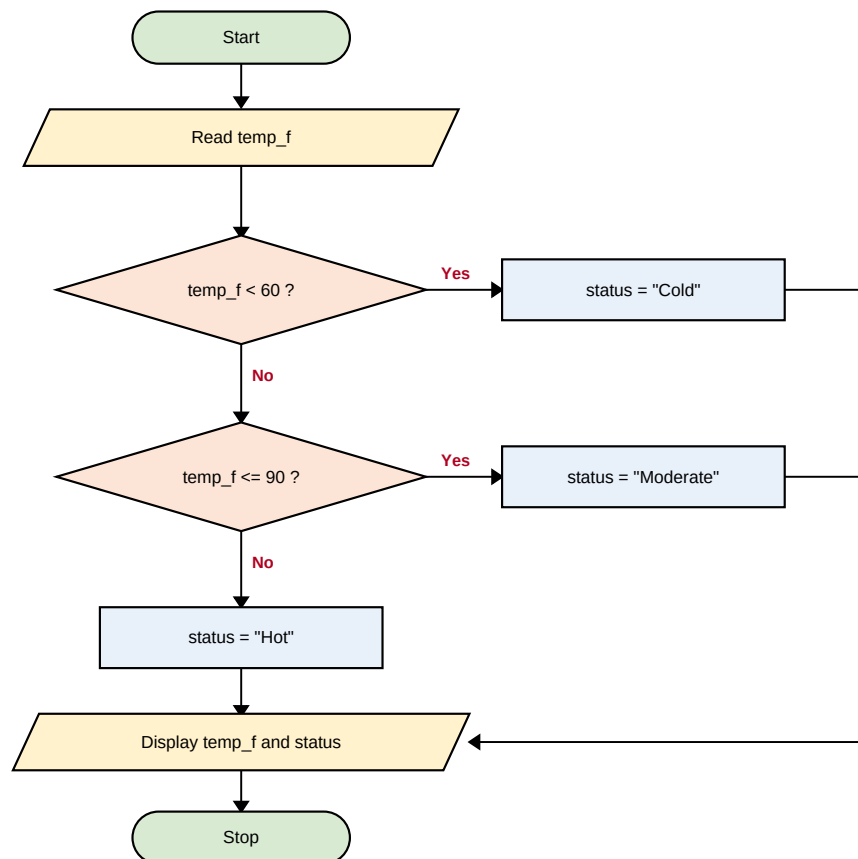
Answer 44 - Set 4 of 10 (temp_f thresholds 60 / 90 (Fahrenheit))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if $\text{temp_f} < 60$, **Moderate** if $60 \leq \text{temp_f} \leq 90$, and **Hot** if $\text{temp_f} > 90$. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **temp_f**.
3. If **temp_f** < 60, set status to **Cold** and go to step 6.
4. Otherwise, if **temp_f** <= 90, set status to **Moderate** and go to step 6.
5. Otherwise ($\text{temp_f} > 90$), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```
temp_f = float(input("Enter the temperature in degrees Fahrenheit: "))
if temp_f < 60:
    status = "Cold"
elif temp_f <= 90:
    status = "Moderate"
else:
    status = "Hot"
print("Temperature :", temp_f, "deg F")
print("Status      :", status)
```

Run 1 : sample input / output

```
Enter the temperature in degrees Fahrenheit: 54.5
Temperature : 54.5 deg F
Status      : Cold
```

Run 2 : sample input / output

```
Enter the temperature in degrees Fahrenheit: 75.5
Temperature : 75.5 deg F
Status      : Moderate
```

Run 3 : sample input / output

```
Enter the temperature in degrees Fahrenheit: 96.5
Temperature : 96.5 deg F
Status      : Hot
```

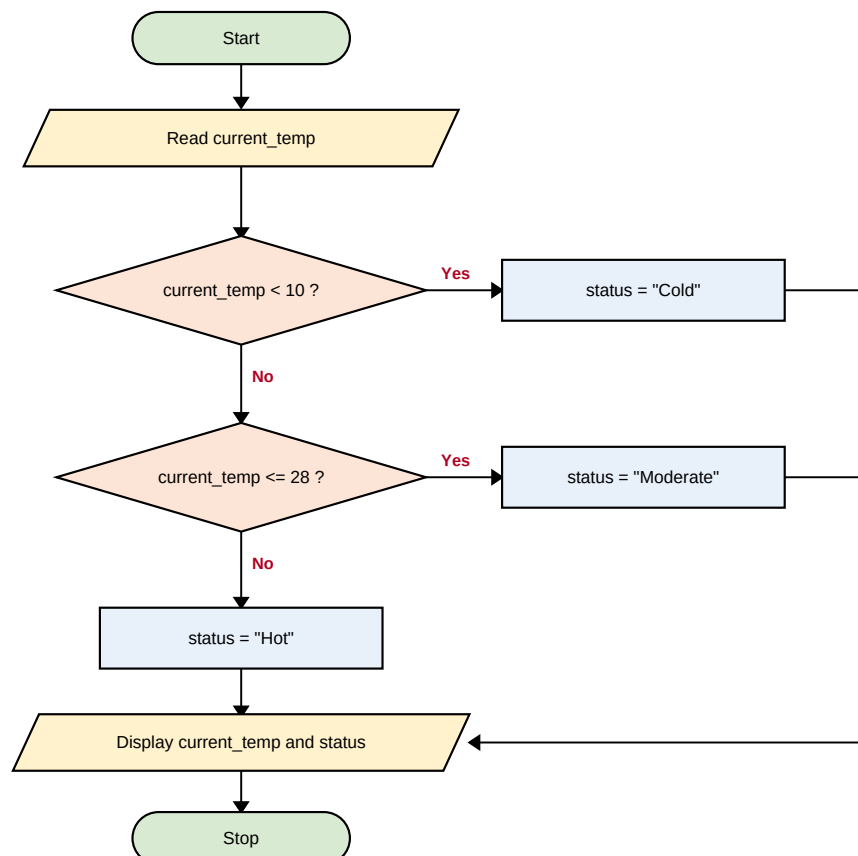
Answer 45 - Set 5 of 10 (current_temp thresholds 10 / 28 (Celsius))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if `current_temp < 10`, **Moderate** if `10 <= current_temp <= 28`, and **Hot** if `current_temp > 28`. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into `current_temp`.
3. If `current_temp < 10`, set status to **Cold** and go to step 6.
4. Otherwise, if `current_temp <= 28`, set status to **Moderate** and go to step 6.
5. Otherwise (`current_temp > 28`), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```
current_temp = int(input("Enter the temperature in degrees Celsius: "))
if current_temp < 10:
    status = "Cold"
elif current_temp <= 28:
    status = "Moderate"
else:
    status = "Hot"
print(f"The temperature {current_temp} deg C is classified as {status}.")
```

Run 1 : sample input / output

```
Enter the temperature in degrees Celsius: 5
The temperature 5 deg C is classified as Cold.
```

Run 2 : sample input / output

```
Enter the temperature in degrees Celsius: 19
The temperature 19 deg C is classified as Moderate.
```

Run 3 : sample input / output

```
Enter the temperature in degrees Celsius: 34
The temperature 34 deg C is classified as Hot.
```

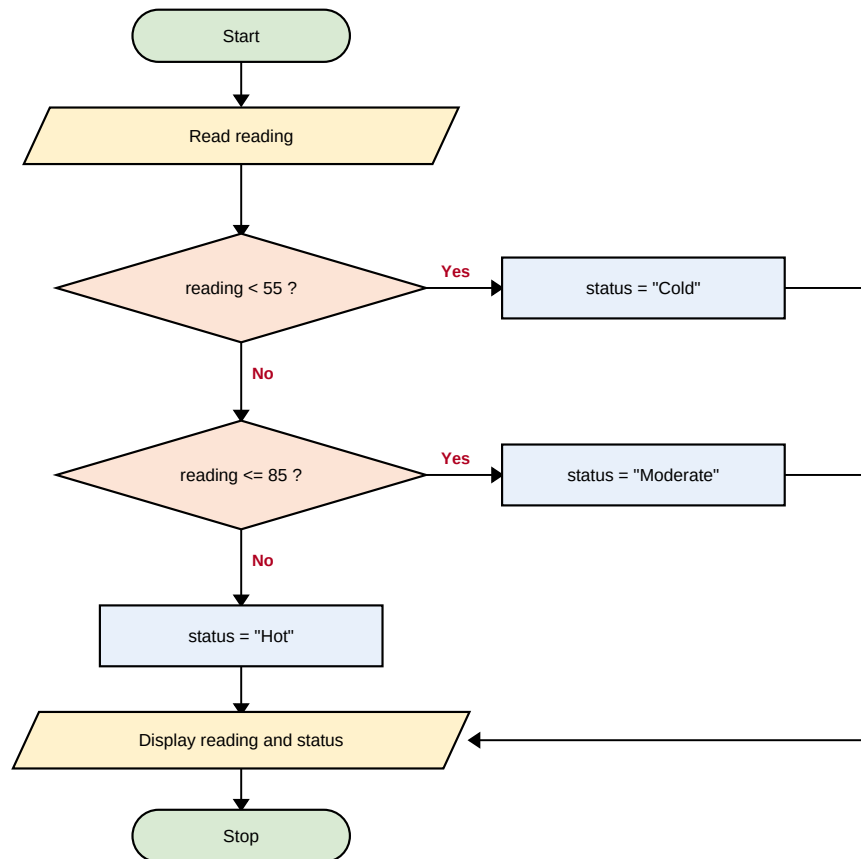
Answer 46 - Set 6 of 10 (reading thresholds 55 / 85 (Fahrenheit))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if reading < 55, **Moderate** if 55 <= reading <= 85, and **Hot** if reading > 85. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **reading**.
3. If **reading < 55**, set status to **Cold** and go to step 6.
4. Otherwise, if **reading <= 85**, set status to **Moderate** and go to step 6.
5. Otherwise (reading > 85), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```

reading = float(input("Enter the temperature in degrees Fahrenheit: "))
if reading < 55:
    status = "Cold"
elif reading <= 85:
    status = "Moderate"
else:
    status = "Hot"
print("Weather alert -> " + status.upper() + " (" + str(reading) + " F)")

```

Run 1 : sample input / output

```

Enter the temperature in degrees Fahrenheit: 49.5
Weather alert -> COLD (49.5 F)

```

Run 2 : sample input / output

```

Enter the temperature in degrees Fahrenheit: 70.5
Weather alert -> MODERATE (70.5 F)

```

Run 3 : sample input / output

```

Enter the temperature in degrees Fahrenheit: 91.5
Weather alert -> HOT (91.5 F)

```

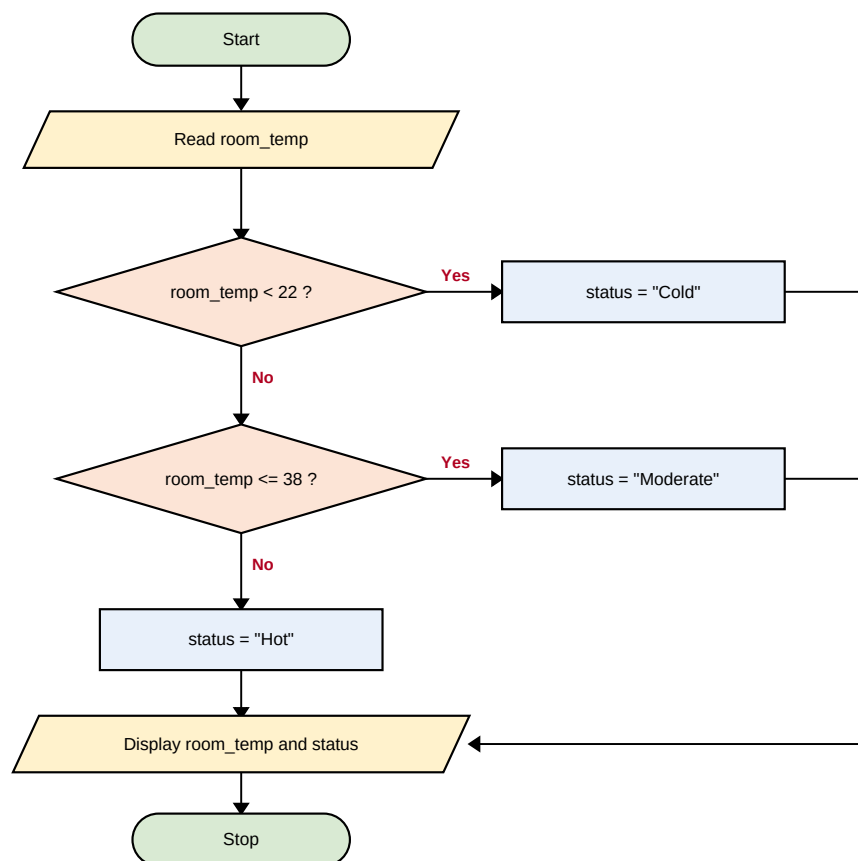
Answer 47 - Set 7 of 10 (room_temp thresholds 22 / 38 (Celsius))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if `room_temp < 22`, **Moderate** if `22 <= room_temp <= 38`, and **Hot** if `room_temp > 38`. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **room_temp**.
3. If **room_temp < 22**, set status to **Cold** and go to step 6.
4. Otherwise, if **room_temp <= 38**, set status to **Moderate** and go to step 6.
5. Otherwise (`room_temp > 38`), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```
room_temp = int(input("Enter the temperature in degrees Celsius: "))
if room_temp < 22:
    status = "Cold"
elif room_temp <= 38:
    status = "Moderate"
else:
    status = "Hot"
print("Temperature :", room_temp, "deg C")
print("Status      :", status)
```

Run 1 : sample input / output

```
Enter the temperature in degrees Celsius: 17
Temperature : 17 deg C
Status      : Cold
```

Run 2 : sample input / output

```
Enter the temperature in degrees Celsius: 30
Temperature : 30 deg C
Status      : Moderate
```

Run 3 : sample input / output

```
Enter the temperature in degrees Celsius: 44
Temperature : 44 deg C
Status      : Hot
```

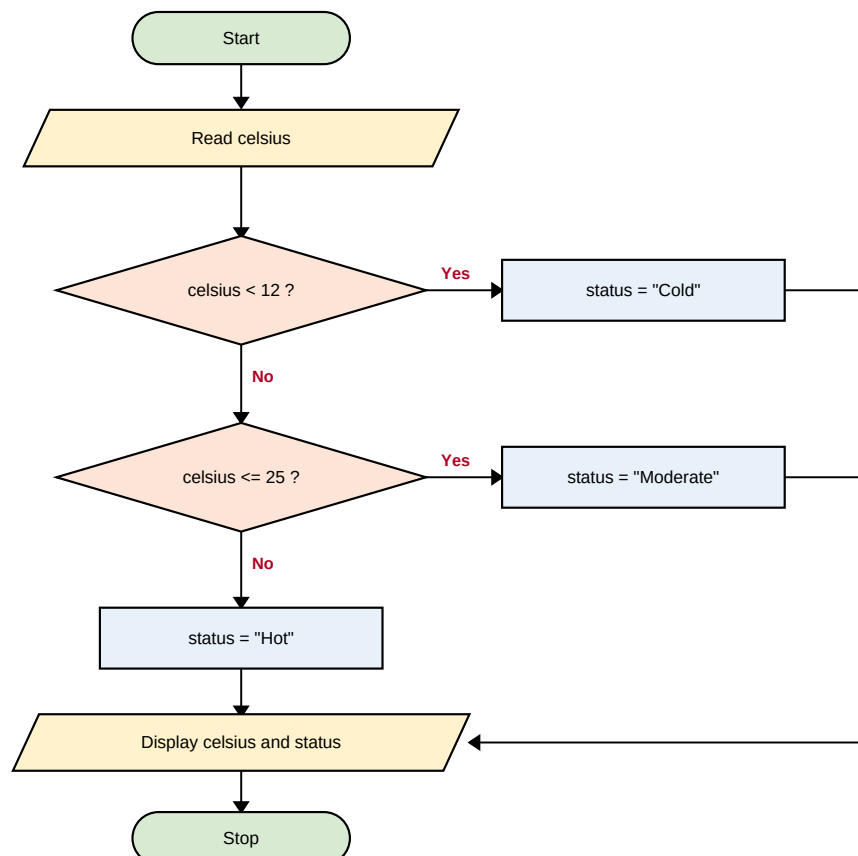
Answer 48 - Set 8 of 10 (celsius thresholds 12 / 25 (Celsius))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if celsius < 12, **Moderate** if 12 <= celsius <= 25, and **Hot** if celsius > 25. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **celsius**.
3. If **celsius < 12**, set status to **Cold** and go to step 6.
4. Otherwise, if **celsius <= 25**, set status to **Moderate** and go to step 6.
5. Otherwise (celsius > 25), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```
celsius = float(input("Enter the temperature in degrees Celsius: "))
if celsius < 12:
    status = "Cold"
elif celsius <= 25:
    status = "Moderate"
else:
    status = "Hot"
print(f"The temperature {celsius} deg C is classified as {status}.")
```

Run 1 : sample input / output

```
Enter the temperature in degrees Celsius: 6.5
The temperature 6.5 deg C is classified as Cold.
```

Run 2 : sample input / output

```
Enter the temperature in degrees Celsius: 19.0
The temperature 19.0 deg C is classified as Moderate.
```

Run 3 : sample input / output

```
Enter the temperature in degrees Celsius: 31.5
The temperature 31.5 deg C is classified as Hot.
```

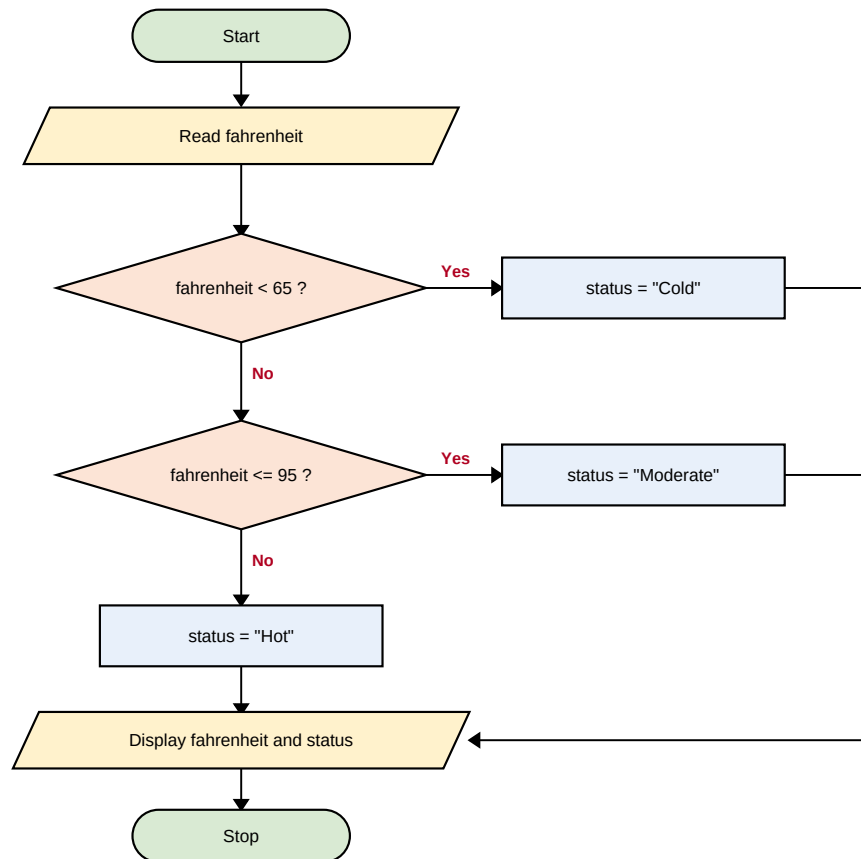
Answer 49 - Set 9 of 10 (fahrenheit thresholds 65 / 95 (Fahrenheit))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if fahrenheit < 65, **Moderate** if 65 <= fahrenheit <= 95, and **Hot** if fahrenheit > 95. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **fahrenheit**.
3. If **fahrenheit < 65**, set status to **Cold** and go to step 6.
4. Otherwise, if **fahrenheit <= 95**, set status to **Moderate** and go to step 6.
5. Otherwise (fahrenheit > 95), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```

fahrenheit = int(input("Enter the temperature in degrees Fahrenheit: "))
if fahrenheit < 65:
    status = "Cold"
elif fahrenheit <= 95:
    status = "Moderate"
else:
    status = "Hot"
print("Weather alert -> " + status.upper() + " (" + str(fahrenheit) + " F)")

```

Run 1 : sample input / output

```

Enter the temperature in degrees Fahrenheit: 60
Weather alert -> COLD (60 F)

```

Run 2 : sample input / output

```

Enter the temperature in degrees Fahrenheit: 80
Weather alert -> MODERATE (80 F)

```

Run 3 : sample input / output

```

Enter the temperature in degrees Fahrenheit: 101
Weather alert -> HOT (101 F)

```

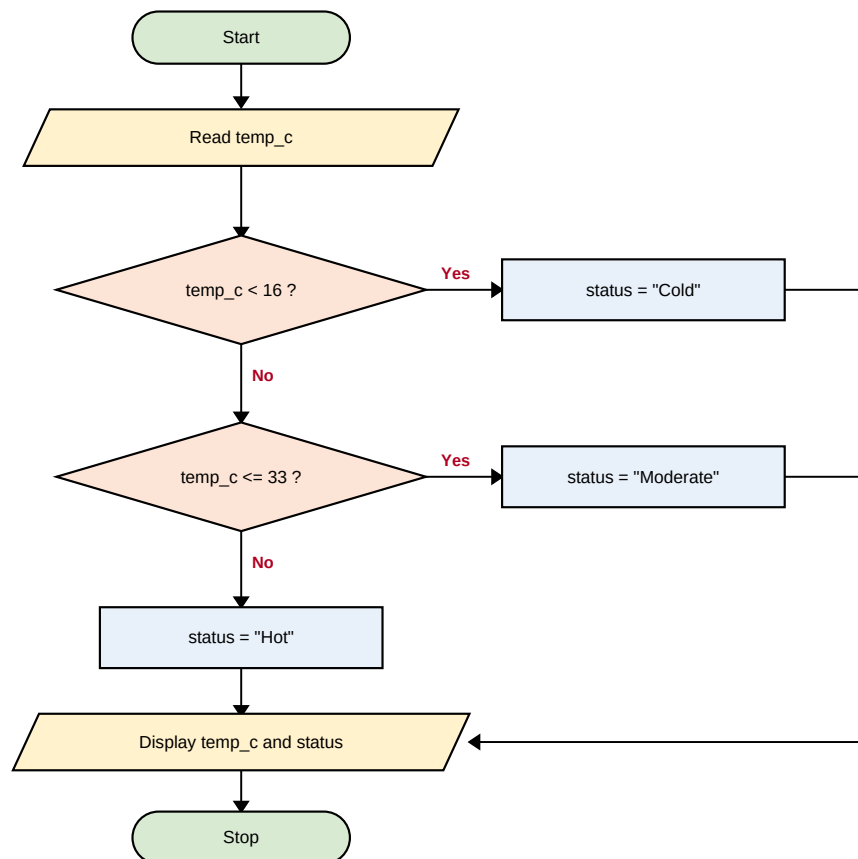
Answer 50 - Set 10 of 10 (temp_c thresholds 16 / 33 (Celsius))

if-elif-else selects exactly one block out of several. Python tests the conditions from top to bottom; the first condition that is True runs its block and the rest are skipped; **else** runs when no condition is True. Here the classification rules are: **Cold** if $\text{temp_c} < 16$, **Moderate** if $16 \leq \text{temp_c} \leq 33$, and **Hot** if $\text{temp_c} > 33$. The **if** checks Cold, the **elif** checks Moderate, and the **else** handles Hot.

Algorithm

1. Start.
2. Read the temperature into **temp_c**.
3. If **temp_c < 16**, set status to **Cold** and go to step 6.
4. Otherwise, if **temp_c ≤ 33**, set status to **Moderate** and go to step 6.
5. Otherwise ($\text{temp_c} > 33$), set status to **Hot**.
6. Display the temperature and its status.
7. Stop.

Flowchart



Program

```
temp_c = float(input("Enter the temperature in degrees Celsius: "))
if temp_c < 16:
    status = "Cold"
elif temp_c <= 33:
    status = "Moderate"
else:
    status = "Hot"
print("Temperature :", temp_c, "deg C")
print("Status      :", status)
```

Run 1 : sample input / output

```
Enter the temperature in degrees Celsius: 10.5
Temperature : 10.5 deg C
Status      : Cold
```

Run 2 : sample input / output

```
Enter the temperature in degrees Celsius: 25.0
Temperature : 25.0 deg C
Status      : Moderate
```

Run 3 : sample input / output

```
Enter the temperature in degrees Celsius: 39.5
Temperature : 39.5 deg C
Status      : Hot
```

Section 6 : Group 2 (Roll Number ends in 1, 6, 8) - Question 3 (Answers 51 - 60)

Question: Explain any five features of Python.

Answer 51 - Set 1 of 10 (Features: Powerful String Handling, Easy File Handling, Object-Oriented Programming ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Powerful String Handling

Strings support slicing, reversing and many built-in methods such as upper(), lower(), replace() and split().

```
word = "python"
print(word.upper())
print(word[::-1])
print(len(word), word[1:4])
```

```
Output:
PYTHON
nohtyp
6 yth
```

2. Easy File Handling

Files can be created, written and read with just a few lines using the with statement, which also closes the file automatically.

```
with open("notes.txt", "w") as f:
    f.write("Rohit is learning Python\n")
with open("notes.txt") as f:
    print(f.read())
```

```
Output:
Rohit is learning Python
```

3. Object-Oriented Programming

Python supports classes and objects, with encapsulation, inheritance and polymorphism, which helps to build reusable and well organised programs.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def show(self):
        print(self.name, "scored", self.marks)
s1 = Student("Rohit", 60)
s1.show()
```

```
Output:
Rohit scored 60
```

4. List Comprehension

A compact one-line syntax builds a new list from a loop and an optional condition, replacing several lines of ordinary loop code.

```
nums = [x for x in range(1, 15) if x % 2 == 0]
print(nums)
```

```
Output:
[2, 4, 6, 8, 10, 12, 14]
```

5. Support for Recursion

A function can call itself, which gives short and elegant solutions for problems such as factorial, Fibonacci and tree traversal.

```
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n - 1)
print("Factorial of 4 =", fact(4))
```

```
Output:
Factorial of 4 = 24
```

Answer 52 - Set 2 of 10 (Features: Dynamically Typed, Multi-Paradigm, Interpreted Language ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Dynamically Typed

A variable does not need a declared type. The type is decided at run time from the assigned value and can change when a new value is assigned.

```
value = 13
print(value, type(value))
value = "program"
print(value, type(value))
value = 13 / 4
print(value, type(value))
```

```
Output:
13 <class 'int'>
program <class 'str'>
3.25 <class 'float'>
```

2. Multi-Paradigm (Functional Style)

Besides procedural and object-oriented styles, Python supports functional programming with lambda, map(), filter() and reduce().

```
nums = [2, 3, 21, 24]
squares = list(map(lambda x: x * x, nums))
print(squares)
```

```
Output:
[4, 9, 441, 576]
```

3. Interpreted Language

Python code is executed line by line by the interpreter. There is no separate compile step, so results appear immediately and testing is quick.

```
print("Step 1: ready")
total = 13 + 67
print("Step 2: total =", total)
print("Step 3: done")
```

```
Output:
Step 1: ready
Step 2: total = 80
Step 3: done
```

4. Multiple Assignment and Tuple Unpacking

Several variables can be assigned in one statement and values can be swapped without a temporary variable.

```
a, b, c = 13, 67, 96
print("Before swap:", a, b, c)
a, b = b, a
print("After swap :", a, b, c)
```

```
Output:
Before swap: 13 67 96
After swap : 67 13 96
```

5. List Comprehension

A compact one-line syntax builds a new list from a loop and an optional condition, replacing several lines of ordinary loop code.

```
nums = [x for x in range(1, 15) if x % 2 == 0]
print(nums)
```

```
Output:
[2, 4, 6, 8, 10, 12, 14]
```

Answer 53 - Set 3 of 10 (Features: Case Sensitive, Arbitrary-Precision Integers, Powerful String Handling ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Case Sensitive

Identifiers are case sensitive: age, Age and AGE are three different variables.

```
age = 19
Age = 11
print(age, Age, age == Age)
```

```
Output:
19 11 False
```

2. Arbitrary-Precision Integers

Integers have no fixed size limit, so very large values can be calculated exactly without overflow.

```
big = 8 ** 38
print(big)
```

```
Output:
20769187434139310514121985316880384
```

3. Powerful String Handling

Strings support slicing, reversing and many built-in methods such as upper(), lower(), replace() and split().

```
word = "computer"
print(word.upper())
print(word[::-1])
print(len(word), word[1:4])
```

```
Output:
COMPUTER
retupmoc
8 omp
```

4. Rich Built-in Data Structures

Lists, tuples, sets and dictionaries are available without importing anything and make storing and processing collections easy.

```
lst = [3, 9, 15, 18]
tup = tuple(lst)
st = set(lst + [11])
dic = {"name": "Karthik", "age": 19}
print(lst)
print(tup)
print(st)
print(dic)
```

```
Output:
[3, 9, 15, 18]
(3, 9, 15, 18)
{3, 9, 11, 15, 18}
{'name': 'Karthik', 'age': 19}
```

5. Extensive Standard Library

Python ships with a large library of ready-made modules (math, datetime, statistics, os and many more) so common tasks need very little code.

```
import math
print("Square root of 81 =", math.sqrt(81))
print("Factorial of 6 =", math.factorial(6))
```

```
Output:
Square root of 81 = 9.0
Factorial of 6 = 720
```

Answer 54 - Set 4 of 10 (Features: Free and Open Source, Multiple Assignment and Tuple Unpacking, Support for Recursion ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Free and Open Source

Python can be downloaded, used and distributed free of charge, and its source code is open, so a large community improves it continuously.

```
import sys
print("Running Python version", sys.version_info.major)
```

```
Output:
Running Python version 3
```

2. Multiple Assignment and Tuple Unpacking

Several variables can be assigned in one statement and values can be swapped without a temporary variable.

```
a, b, c = 32, 69, 84
print("Before swap:", a, b, c)
a, b = b, a
print("After swap :", a, b, c)
```

```
Output:
Before swap: 32 69 84
After swap : 69 32 84
```

3. Support for Recursion

A function can call itself, which gives short and elegant solutions for problems such as factorial, Fibonacci and tree traversal.

```
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n - 1)
print("Factorial of 7 =", fact(7))
```

```
Output:
Factorial of 7 = 5040
```

4. Rich Built-in Data Structures

Lists, tuples, sets and dictionaries are available without importing anything and make storing and processing collections easy.

```
lst = [4, 12, 17, 18]
tup = tuple(lst)
st = set(lst + [32])
dic = {"name": "Sana", "age": 24}
print(lst)
print(tup)
print(st)
print(dic)
```

```
Output:
[4, 12, 17, 18]
(4, 12, 17, 18)
```

```
{32, 4, 12, 17, 18}
{'name': 'Sana', 'age': 24}
```

5. Easy Output Formatting (f-strings)

Formatted strings place the values of variables and expressions directly inside text, which makes output code short and readable.

```
item = "internet"
price = 84
print(f"{item} costs Rs. {price * 2} for two")
```

```
Output:
internet costs Rs. 168 for two
```

Answer 55 - Set 5 of 10 (Features: Powerful String Handling, Object-Oriented Programming, Easy Output Formatting ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Powerful String Handling

Strings support slicing, reversing and many built-in methods such as upper(), lower(), replace() and split().

```
word = "keyboard"
print(word.upper())
print(word[::-1])
print(len(word), word[1:4])
```

```
Output:
KEYBOARD
draobyek
8 eyb
```

2. Object-Oriented Programming

Python supports classes and objects, with encapsulation, inheritance and polymorphism, which helps to build reusable and well organised programs.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def show(self):
        print(self.name, "scored", self.marks)
s1 = Student("Vikram", 62)
s1.show()
```

```
Output:
Vikram scored 62
```

3. Easy Output Formatting (f-strings)

Formatted strings place the values of variables and expressions directly inside text, which makes output code short and readable.

```
item = "keyboard"
price = 62
print(f"{item} costs Rs. {price * 2} for two")
```

```
Output:
keyboard costs Rs. 124 for two
```

4. Arbitrary-Precision Integers

Integers have no fixed size limit, so very large values can be calculated exactly without overflow.

```
big = 5 ** 45
print(big)
```

```
Output:
28421709430404007434844970703125
```

5. Multiple Assignment and Tuple Unpacking

Several variables can be assigned in one statement and values can be swapped without a temporary variable.

```
a, b, c = 35, 81, 62
print("Before swap:", a, b, c)
a, b = b, a
print("After swap :", a, b, c)
```

```
Output:
Before swap: 35 81 62
After swap : 81 35 62
```

Answer 56 - Set 6 of 10 (Features: Free and Open Source, List Comprehension, Multiple Assignment and Tuple Unpacking ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Free and Open Source

Python can be downloaded, used and distributed free of charge, and its source code is open, so a large community improves it continuously.

```
import sys
print("Running Python version", sys.version_info.major)
```

```
Output:
Running Python version 3
```

2. List Comprehension

A compact one-line syntax builds a new list from a loop and an optional condition, replacing several lines of ordinary loop code.

```
nums = [x for x in range(1, 11) if x % 2 == 0]
print(nums)
```

```
Output:
[2, 4, 6, 8, 10]
```

3. Multiple Assignment and Tuple Unpacking

Several variables can be assigned in one statement and values can be swapped without a temporary variable.

```
a, b, c = 22, 63, 85
print("Before swap:", a, b, c)
a, b = b, a
print("After swap :", a, b, c)
```

```
Output:
Before swap: 22 63 85
After swap : 63 22 85
```

4. Powerful String Handling

Strings support slicing, reversing and many built-in methods such as upper(), lower(), replace() and split().

```
word = "software"
print(word.upper())
print(word[::-1])
print(len(word), word[1:4])
```

```
Output:
SOFTWARE
erawtfos
8 oft
```

5. Strongly Typed

Python does not silently mix incompatible types. Adding a string and an integer raises a TypeError until one of them is converted explicitly.

```
try:
```

```
print("22" + 63)
except TypeError:
    print("Cannot add str and int without conversion")
print(int("22") + 63)
```

```
Output:
Cannot add str and int without conversion
85
```

Answer 57 - Set 7 of 10 (Features: Support for Recursion, Dynamically Typed, Rich Built-in Data Structures ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Support for Recursion

A function can call itself, which gives short and elegant solutions for problems such as factorial, Fibonacci and tree traversal.

```
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n - 1)
print("Factorial of 5 =", fact(5))
```

```
Output:
Factorial of 5 = 120
```

2. Dynamically Typed

A variable does not need a declared type. The type is decided at run time from the assigned value and can change when a new value is assigned.

```
value = 16
print(value, type(value))
value = "variable"
print(value, type(value))
value = 16 / 4
print(value, type(value))
```

```
Output:
16 <class 'int'>
variable <class 'str'>
4.0 <class 'float'>
```

3. Rich Built-in Data Structures

Lists, tuples, sets and dictionaries are available without importing anything and make storing and processing collections easy.

```
lst = [6, 11, 18, 22]
tup = tuple(lst)
st = set(lst + [16])
dic = {"name": "Naveen", "age": 27}
print(lst)
print(tup)
print(st)
print(dic)
```

```
Output:
[6, 11, 18, 22]
(6, 11, 18, 22)
{6, 11, 16, 18, 22}
{'name': 'Naveen', 'age': 27}
```

4. Portable (Platform Independent)

The same Python program runs on Windows, Linux and macOS without modification, because the interpreter handles the platform differences.

```
import os
path = os.path.join("variable", "notes.txt")
```

```
print(path) # prints variable\notes.txt on Windows
```

```
Output:  
variable/notes.txt
```

5. Simple and Easy to Learn

Python syntax is close to plain English, uses no semicolons or braces and needs very few lines of code, so programs are easy to read, write and maintain.

```
name = "Naveen"  
print("Hello,", name)  
print("Square of 16 is", 16 * 16)
```

```
Output:  
Hello, Naveen  
Square of 16 is 256
```

Answer 58 - Set 8 of 10 (Features: Dynamically Typed, Arbitrary-Precision Integers, High-Level Language ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Dynamically Typed

A variable does not need a declared type. The type is decided at run time from the assigned value and can change when a new value is assigned.

```
value = 14  
print(value, type(value))  
value = "function"  
print(value, type(value))  
value = 14 / 4  
print(value, type(value))
```

```
Output:  
14 <class 'int'>  
function <class 'str'>  
3.5 <class 'float'>
```

2. Arbitrary-Precision Integers

Integers have no fixed size limit, so very large values can be calculated exactly without overflow.

```
big = 7 ** 26  
print(big)
```

```
Output:  
9387480337647754305649
```

3. High-Level Language

Programmers do not deal with memory addresses or machine instructions. Built-in types such as lists with indexing and slicing hide low-level details.

```
fruits = ['cherry', 'mango', 'grapes']  
print(fruits[0], fruits[-1], len(fruits))
```

```
Output:  
cherry grapes 3
```

4. Automatic Memory Management

Memory is allocated and released automatically by Python's garbage collector, so the programmer does not call malloc() or free().

```
data = [14, 55, 83]  
print("Before:", data)  
data = None # old list is freed automatically  
print("After:", data)
```

```
Output:
```

```
Before: [14, 55, 83]
After: None
```

5. Easy Output Formatting (f-strings)

Formatted strings place the values of variables and expressions directly inside text, which makes output code short and readable.

```
item = "function"
price = 83
print(f"{item} costs Rs. {price * 2} for two")
```

```
Output:
function costs Rs. 166 for two
```

Answer 59 - Set 9 of 10 (Features: Case Sensitive, Portable, Exception Handling ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. Case Sensitive

Identifiers are case sensitive: age, Age and AGE are three different variables.

```
age = 28
Age = 9
print(age, Age, age == Age)
```

```
Output:
28 9 False
```

2. Portable (Platform Independent)

The same Python program runs on Windows, Linux and macOS without modification, because the interpreter handles the platform differences.

```
import os
path = os.path.join("language", "notes.txt")
print(path) # prints language\notes.txt on Windows
```

```
Output:
language/notes.txt
```

3. Exception Handling

Runtime errors can be caught with try, except and finally blocks, so a program can handle problems gracefully instead of crashing.

```
try:
    result = 9 / 0
except ZeroDivisionError:
    print("Cannot divide 9 by zero")
finally:
    print("Program continues")
```

```
Output:
Cannot divide 9 by zero
Program continues
```

4. Interpreted Language

Python code is executed line by line by the interpreter. There is no separate compile step, so results appear immediately and testing is quick.

```
print("Step 1: ready")
total = 9 + 44
print("Step 2: total =", total)
print("Step 3: done")
```

```
Output:
Step 1: ready
Step 2: total = 53
Step 3: done
```

5. Large Third-Party Library Support

Thousands of packages for data science, web, AI and GUI development can be installed with pip (for example numpy and pandas).

```
# pip install numpy (run once in the terminal)
import statistics
marks = [5, 13, 20, 23]
print("Mean =", statistics.mean(marks))
print("Highest =", max(marks))
```

```
Output:
Mean = 15.25
Highest = 23
```

Answer 60 - Set 10 of 10 (Features: High-Level Language, Simple and Easy to Learn, Free and Open Source ...)

Python is a popular, general purpose, high-level language. Five of its important features, each with a short program and its output, are explained below.

1. High-Level Language

Programmers do not deal with memory addresses or machine instructions. Built-in types such as lists with indexing and slicing hide low-level details.

```
fruits = ['melon', 'banana', 'papaya']
print(fruits[0], fruits[-1], len(fruits))
```

```
Output:
melon papaya 3
```

2. Simple and Easy to Learn

Python syntax is close to plain English, uses no semicolons or braces and needs very few lines of code, so programs are easy to read, write and maintain.

```
name = "Bhavna"
print("Hello,", name)
print("Square of 36 is", 36 * 36)
```

```
Output:
Hello, Bhavna
Square of 36 is 1296
```

3. Free and Open Source

Python can be downloaded, used and distributed free of charge, and its source code is open, so a large community improves it continuously.

```
import sys
print("Running Python version", sys.version_info.major)
```

```
Output:
Running Python version 3
```

4. Interpreted Language

Python code is executed line by line by the interpreter. There is no separate compile step, so results appear immediately and testing is quick.

```
print("Step 1: ready")
total = 36 + 71
print("Step 2: total =", total)
print("Step 3: done")
```

```
Output:
Step 1: ready
Step 2: total = 107
Step 3: done
```

5. Easy Output Formatting (f-strings)

Formatted strings place the values of variables and expressions directly inside text, which makes output code short and readable.

```
item = "developer"  
price = 60  
print(f"{item} costs Rs. {price * 2} for two")
```

```
Output:  
developer costs Rs. 120 for two
```

Section 7 : Group 3 (Roll Number ends in 2, 4, 9) - Question 1 (Answers 61 - 70)

Question: Explain logical operators and bitwise AND, OR, NOT in Python with suitable examples. An online shopping system provides a discount when the purchase amount is greater than Rs. 5,000 and the customer has a membership. Identify the logical operator required and write the corresponding condition.

Answer 61 - Set 1 of 10 (Discount check for Rs. 7500, member: yes)

Logical operators (and, or, not) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators (&, |, ~)** work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 12 and 10

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	12 & 10	00001100 & 00001010 = 00001000 = 8
(OR)	1 if at least one bit is 1	12 10	00001100 00001010 = 00001110 = 14
~ (NOT)	Inverts every bit: ~x = -(x + 1)	~12	~00001100 = -13

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 and the customer is a member), so the **logical and** operator is required. Condition: **if purchase_amount > 5000 and is_member:**. For Rs. 7500 and member = yes the condition is True.

Program

```
purchase_amount = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
is_member = reply.lower() == "yes"
if purchase_amount > 5000 and is_member:
    discount = purchase_amount * 10 / 100
    print("Discount of Rs.", discount, "applied")
    print("Amount to pay: Rs.", purchase_amount - discount)
else:
    print("No discount available")
    print("Amount to pay: Rs.", purchase_amount)
# Bitwise AND, OR and NOT
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("a & b =", a & b)
print("a | b =", a | b)
print("~a =", ~a)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 7500
Is the customer a member? (yes/no): yes
Discount of Rs. 750.0 applied
Amount to pay: Rs. 6750.0
Enter first number: 12
Enter second number: 10
a & b = 8
a | b = 14
~a = -13
```

Answer 62 - Set 2 of 10 (Discount check for Rs. 3200, member: yes)

Logical operators (**and**, **or**, **not**) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators** (&, |, ~) work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 25 and 9

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	25 & 9	00011001 & 00001001 = 00001001 = 9
(OR)	1 if at least one bit is 1	25 9	00011001 00001001 = 00011001 = 25
~ (NOT)	Inverts every bit: $\sim x = -(x + 1)$	~25	~00011001 = -26

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if bill > 5000 and has_membership:**. For Rs. 3200 and member = yes the condition is False.

Program

```
bill = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
has_membership = reply.lower() == "yes"
if bill > 5000 and has_membership:
    discount = bill * 15 / 100
    print(f"You saved Rs. {discount:.2f}")
    print(f"Final bill = Rs. {bill - discount:.2f}")
else:
    print("Discount condition not satisfied")
    print(f"Final bill = Rs. {bill:.2f}")
# Bitwise AND, OR and NOT
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))
print("x & y =", x & y)
print("x | y =", x | y)
print("~x =", ~x)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 3200
Is the customer a member? (yes/no): yes
Discount condition not satisfied
Final bill = Rs. 3200.00
Enter first number: 25
Enter second number: 9
x & y = 9
x | y = 25
~x = -26
```

Answer 63 - Set 3 of 10 (Discount check for Rs. 5000, member: yes)

Logical operators (**and**, **or**, **not**) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators** (&, |, ~) work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 6 and 3

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	6 & 3	00000110 & 00000011 = 00000010 = 2
(OR)	1 if at least one bit is 1	6 3	00000110 00000011 = 00000111 = 7
~ (NOT)	Inverts every bit: $\sim x = -(x + 1)$	~6	~00000110 = -7

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if cart_total > 5000 and member:**. For Rs. 5000 and member = yes the condition is False.

Program

```
cart_total = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
member = reply.lower() == "yes"
if cart_total > 5000 and member:
    discount = cart_total * 20 / 100
    print("Discount of Rs.", discount, "applied")
    print("Amount to pay: Rs.", cart_total - discount)
else:
    print("No discount available")
    print("Amount to pay: Rs.", cart_total)
# Bitwise AND, OR and NOT
p = int(input("Enter first number: "))
q = int(input("Enter second number: "))
print("p & q =", p & q)
print("p | q =", p | q)
print("~p =", ~p)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 5000
Is the customer a member? (yes/no): yes
No discount available
Amount to pay: Rs. 5000
Enter first number: 6
Enter second number: 3
p & q = 2
p | q = 7
~p = -7
```

Answer 64 - Set 4 of 10 (Discount check for Rs. 12000, member: no)

Logical operators (**and**, **or**, **not**) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators** (&, |, ~) work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 14 and 7

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	14 & 7	00001110 & 00000111 = 00000110 = 6
(OR)	1 if at least one bit is 1	14 7	00001110 00000111 = 00001111 = 15
~ (NOT)	Inverts every bit: ~x = -(x + 1)	~14	~00001110 = -15

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if total_cost > 5000 and membership:**. For Rs. 12000 and member = no the condition is False.

Program

```
total_cost = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
membership = reply.lower() == "yes"
if total_cost > 5000 and membership:
    discount = total_cost * 5 / 100
    print(f"You saved Rs. {discount:.2f}")
    print(f"Final bill = Rs. {total_cost - discount:.2f}")
else:
    print("Discount condition not satisfied")
    print(f"Final bill = Rs. {total_cost:.2f}")
# Bitwise AND, OR and NOT
m = int(input("Enter first number: "))
n = int(input("Enter second number: "))
print("m & n =", m & n)
print("m | n =", m | n)
print("~m =", ~m)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 12000
Is the customer a member? (yes/no): no
Discount condition not satisfied
Final bill = Rs. 12000.00
Enter first number: 14
Enter second number: 7
m & n = 6
m | n = 15
~m = -15
```

Answer 65 - Set 5 of 10 (Discount check for Rs. 6400, member: yes)

Logical operators (**and**, **or**, **not**) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators** (**&**, **|**, **~**) work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 21 and 13

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	21 & 13	00010101 & 00001101 = 00000101 = 5
(OR)	1 if at least one bit is 1	21 13	00010101 00001101 = 00011101 = 29
~ (NOT)	Inverts every bit: $\sim x = -(x + 1)$	~21	~00010101 = -22

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if amt > 5000 and mem:.** For Rs. 6400 and member = yes the condition is True.

Program

```
amt = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
mem = reply.lower() == "yes"
if amt > 5000 and mem:
    discount = amt * 12 / 100
    print("Discount of Rs.", discount, "applied")
    print("Amount to pay: Rs.", amt - discount)
else:
    print("No discount available")
    print("Amount to pay: Rs.", amt)
# Bitwise AND, OR and NOT
n1 = int(input("Enter first number: "))
n2 = int(input("Enter second number: "))
print("n1 & n2 =", n1 & n2)
print("n1 | n2 =", n1 | n2)
print("~n1 =", ~n1)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 6400
Is the customer a member? (yes/no): yes
Discount of Rs. 768.0 applied
Amount to pay: Rs. 5632.0
Enter first number: 21
Enter second number: 13
n1 & n2 = 5
n1 | n2 = 29
~n1 = -22
```

Answer 66 - Set 6 of 10 (Discount check for Rs. 4999, member: yes)

Logical operators (**and**, **or**, **not**) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators** (**&**, **|**, **~**) work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 9 and 12

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	9 & 12	00001001 & 00001100 = 00001000 = 8
(OR)	1 if at least one bit is 1	9 12	00001001 00001100 = 00001101 = 13
~ (NOT)	Inverts every bit: $\sim x = -(x + 1)$	~9	~00001001 = -10

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if order_value > 5000 and is_premium:**. For Rs. 4999 and member = yes the condition is False.

Program

```
order_value = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
is_premium = reply.lower() == "yes"
if order_value > 5000 and is_premium:
    discount = order_value * 10 / 100
    print(f"You saved Rs. {discount:.2f}")
    print(f"Final bill = Rs. {order_value - discount:.2f}")
else:
    print("Discount condition not satisfied")
    print(f"Final bill = Rs. {order_value:.2f}")
# Bitwise AND, OR and NOT
u = int(input("Enter first number: "))
v = int(input("Enter second number: "))
print("u & v =", u & v)
print("u | v =", u | v)
print("~u =", ~u)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 4999
Is the customer a member? (yes/no): yes
Discount condition not satisfied
Final bill = Rs. 4999.00
Enter first number: 9
Enter second number: 12
u & v = 8
u | v = 13
~u = -10
```

Answer 67 - Set 7 of 10 (Discount check for Rs. 5001, member: yes)

Logical operators (**and**, **or**, **not**) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators** (&, |, ~) work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 30 and 17

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	30 & 17	00011110 & 00010001 = 00010000 = 16
(OR)	1 if at least one bit is 1	30 17	00011110 00010001 = 00011111 = 31
~ (NOT)	Inverts every bit: ~x = -(x + 1)	~30	~00011110 = -31

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if price > 5000 and club_member:**. For Rs. 5001 and member = yes the condition is True.

Program

```
price = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
club_member = reply.lower() == "yes"
if price > 5000 and club_member:
    discount = price * 25 / 100
    print("Discount of Rs.", discount, "applied")
    print("Amount to pay: Rs.", price - discount)
else:
    print("No discount available")
    print("Amount to pay: Rs.", price)
# Bitwise AND, OR and NOT
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print("num1 & num2 =", num1 & num2)
print("num1 | num2 =", num1 | num2)
print("~num1 =", ~num1)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 5001
Is the customer a member? (yes/no): yes
Discount of Rs. 1250.25 applied
Amount to pay: Rs. 3750.75
Enter first number: 30
Enter second number: 17
num1 & num2 = 16
num1 | num2 = 31
~num1 = -31
```

Answer 68 - Set 8 of 10 (Discount check for Rs. 15000, member: yes)

Logical operators (**and**, **or**, **not**) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators** (**&**, **|**, **~**) work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 11 and 5

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	11 & 5	00001011 & 00000101 = 00000001 = 1
(OR)	1 if at least one bit is 1	11 5	00001011 00000101 = 00001111 = 15
~ (NOT)	Inverts every bit: $\sim x = -(x + 1)$	~11	~00001011 = -12

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if net_amount > 5000 and has_card:**. For Rs. 15000 and member = yes the condition is True.

Program

```
net_amount = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
has_card = reply.lower() == "yes"
if net_amount > 5000 and has_card:
    discount = net_amount * 8 / 100
    print(f"You saved Rs. {discount:.2f}")
    print(f"Final bill = Rs. {net_amount - discount:.2f}")
else:
    print("Discount condition not satisfied")
    print(f"Final bill = Rs. {net_amount:.2f}")
# Bitwise AND, OR and NOT
s = int(input("Enter first number: "))
t = int(input("Enter second number: "))
print("s & t =", s & t)
print("s | t =", s | t)
print("~s =", ~s)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 15000
Is the customer a member? (yes/no): yes
You saved Rs. 1200.00
Final bill = Rs. 13800.00
Enter first number: 11
Enter second number: 5
s & t = 1
s | t = 15
~s = -12
```

Answer 69 - Set 9 of 10 (Discount check for Rs. 8800, member: no)

Logical operators (**and**, **or**, **not**) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators** (&, |, ~) work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 27 and 22

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	27 & 22	00011011 & 00010110 = 00010010 = 18
(OR)	1 if at least one bit is 1	27 22	00011011 00010110 = 00011111 = 31
~ (NOT)	Inverts every bit: $\sim x = -(x + 1)$	~27	~00011011 = -28

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if subtotal > 5000 and is_prime:**. For Rs. 8800 and member = no the condition is False.

Program

```
subtotal = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
is_prime = reply.lower() == "yes"
if subtotal > 5000 and is_prime:
    discount = subtotal * 15 / 100
    print("Discount of Rs.", discount, "applied")
    print("Amount to pay: Rs.", subtotal - discount)
else:
    print("No discount available")
    print("Amount to pay: Rs.", subtotal)
# Bitwise AND, OR and NOT
val1 = int(input("Enter first number: "))
val2 = int(input("Enter second number: "))
print("val1 & val2 =", val1 & val2)
print("val1 | val2 =", val1 | val2)
print("~val1 =", ~val1)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 8800
Is the customer a member? (yes/no): no
No discount available
Amount to pay: Rs. 8800
Enter first number: 27
Enter second number: 22
val1 & val2 = 18
val1 | val2 = 31
~val1 = -28
```

Answer 70 - Set 10 of 10 (Discount check for Rs. 2500, member: no)

Logical operators (and, or, not) combine Boolean conditions and return True or False: **and** is True only if both operands are True, **or** is True if at least one operand is True, and **not** reverses the truth value. **Bitwise operators (&, |, ~)** work on the individual bits of integers.

Truth table of logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Bitwise AND, OR and NOT with 18 and 6

Operator	Description	Example	Working
& (AND)	1 only if both bits are 1	18 & 6	00010010 & 00000110 = 00000010 = 2
(OR)	1 if at least one bit is 1	18 6	00010010 00000110 = 00010110 = 22
~ (NOT)	Inverts every bit: $\sim x = -(x + 1)$	~18	~00010010 = -19

Online shopping discount: the discount needs two conditions to be true together (amount above Rs. 5,000 *and* the customer is a member), so the **logical and** operator is required. Condition: **if payable > 5000 and registered:.** For Rs. 2500 and member = no the condition is False.

Program

```
payable = int(input("Enter the purchase amount in Rs.: "))
reply = input("Is the customer a member? (yes/no): ")
registered = reply.lower() == "yes"
if payable > 5000 and registered:
    discount = payable * 20 / 100
    print(f"You saved Rs. {discount:.2f}")
    print(f"Final bill = Rs. {payable - discount:.2f}")
else:
    print("Discount condition not satisfied")
    print(f"Final bill = Rs. {payable:.2f}")
# Bitwise AND, OR and NOT
first = int(input("Enter first number: "))
second = int(input("Enter second number: "))
print("first & second =", first & second)
print("first | second =", first | second)
print("~first =", ~first)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter the purchase amount in Rs.: 2500
Is the customer a member? (yes/no): no
Discount condition not satisfied
Final bill = Rs. 2500.00
Enter first number: 18
Enter second number: 6
first & second = 2
first | second = 22
~first = -19
```

Section 8 : Group 3 (Roll Number ends in 2, 4, 9) - Question 2 (Answers 71 - 80)

Question: A college wants to count how many students have passed out of 20 students. Write a Python program using a loop and decision-making statement. Represent Algorithm and flowchart.

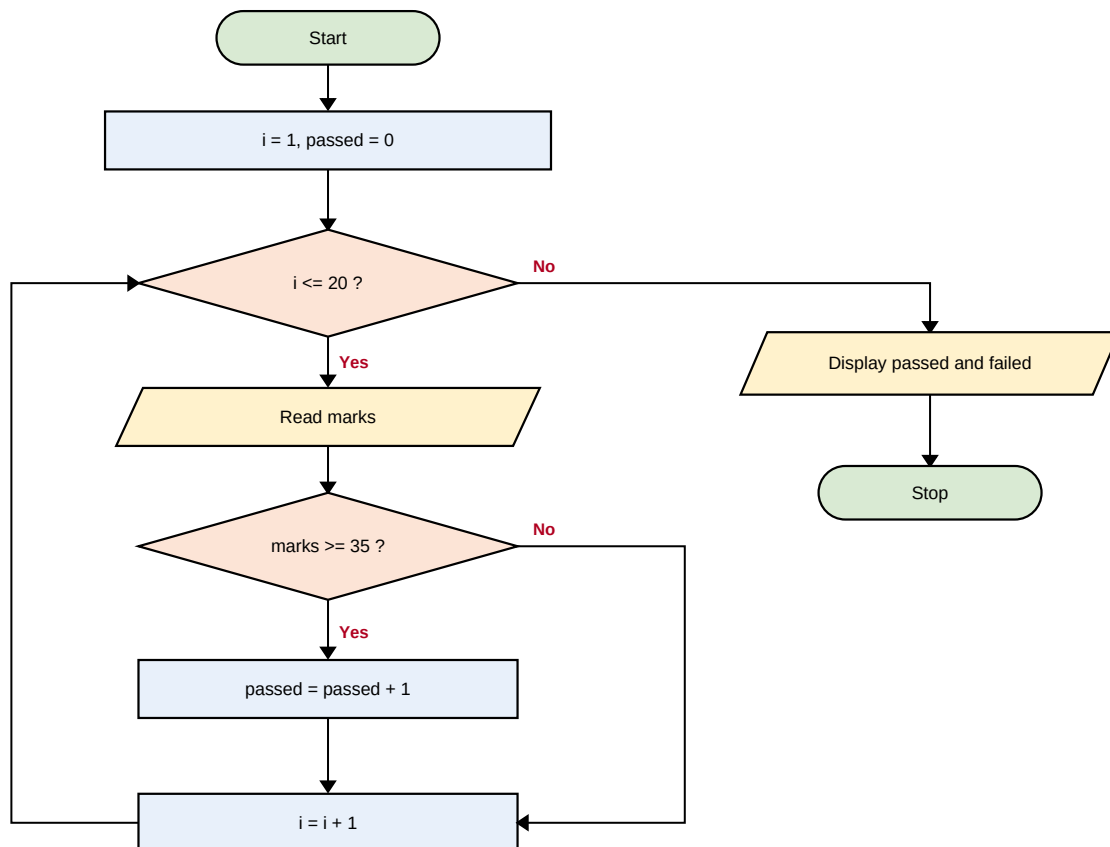
Answer 71 - Set 1 of 10 (Pass mark 35, for loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (35). Each time the condition is True the counter **passed** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **passed = 0** and the loop variable ($i = 1$).
3. Repeat steps 4 to 6 while $i \leq 20$ (once for each of the 20 students).
4. Read the marks of the student into **marks**.
5. If **marks ≥ 35** , the student has passed, so add 1: **passed = passed + 1**.
6. Increase the loop variable: $i = i + 1$.
7. Display the number of students passed (**passed**) and failed ($20 - \text{passed}$).
8. Stop.

Flowchart



Program

```
passed = 0
failed = 0
for i in range(1, 21):
    marks = int(input("Enter marks of student " + str(i) + ": "))
    if marks >= 35:
        passed = passed + 1
    else:
        failed += 1
print("Number of students passed:", passed)
print("Number of students failed:", failed)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter marks of student 1: 73
Enter marks of student 2: 81
Enter marks of student 3: 88
Enter marks of student 4: 73
Enter marks of student 5: 46
Enter marks of student 6: 95
Enter marks of student 7: 62
Enter marks of student 8: 41
Enter marks of student 9: 27
Enter marks of student 10: 57
Enter marks of student 11: 87
Enter marks of student 12: 25
Enter marks of student 13: 16
Enter marks of student 14: 39
Enter marks of student 15: 56
Enter marks of student 16: 43
Enter marks of student 17: 64
Enter marks of student 18: 51
Enter marks of student 19: 20
Enter marks of student 20: 22
Number of students passed: 15
Number of students failed: 5
```

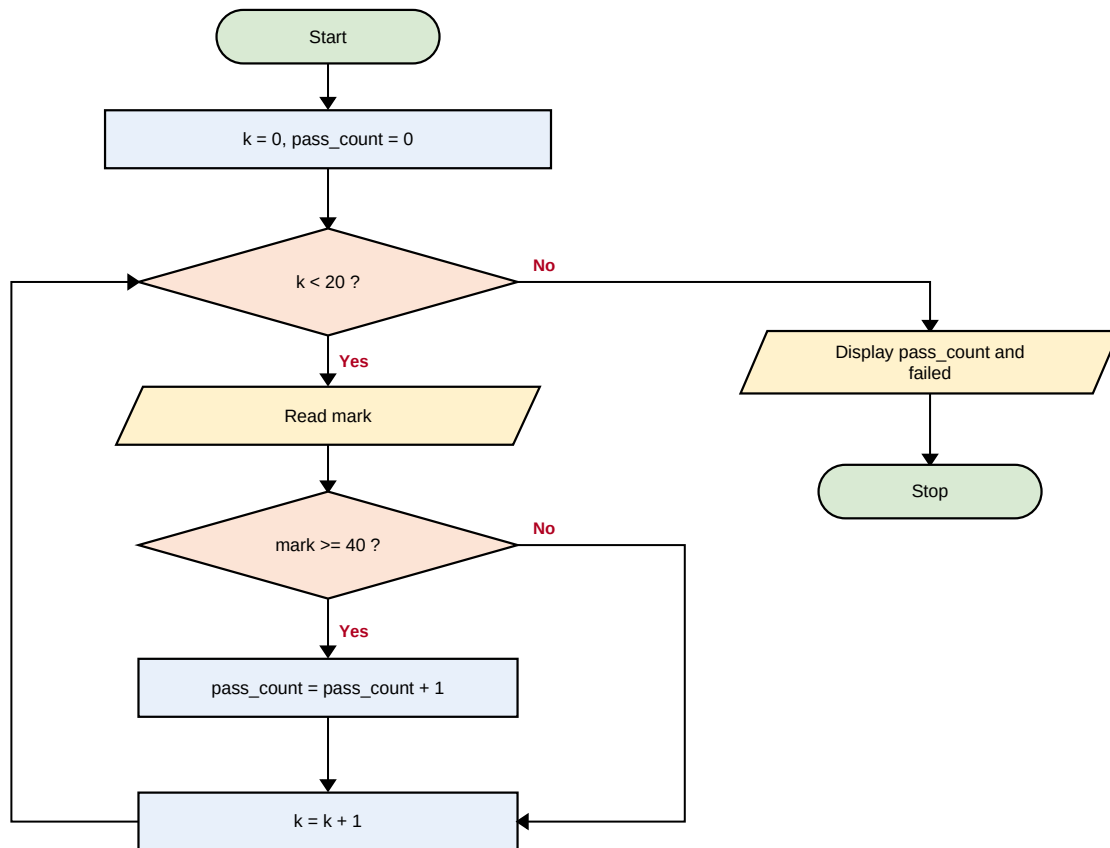
Answer 72 - Set 2 of 10 (Pass mark 40, for loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (40). Each time the condition is True the counter **pass_count** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **pass_count = 0** and the loop variable (**k = 0**).
3. Repeat steps 4 to 6 while **k < 20** (once for each of the 20 students).
4. Read the marks of the student into **mark**.
5. If **mark >= 40**, the student has passed, so add 1: **pass_count = pass_count + 1**.
6. Increase the loop variable: **k = k + 1**.
7. Display the number of students passed (**pass_count**) and failed (**20 - pass_count**).
8. Stop.

Flowchart



Program

```

pass_count = 0
for k in range(20):
    mark = int(input(f"Enter marks of student {k + 1}: "))
    if mark >= 40:
        pass_count = pass_count + 1
print("Number of students passed:", pass_count)
print("Number of students failed:", 20 - pass_count)

```

Sample Input / Output (typed values appear after each prompt)

```

Enter marks of student 1: 95
Enter marks of student 2: 50
Enter marks of student 3: 94
Enter marks of student 4: 59
Enter marks of student 5: 44
Enter marks of student 6: 95
Enter marks of student 7: 55
Enter marks of student 8: 57
Enter marks of student 9: 81
Enter marks of student 10: 60
Enter marks of student 11: 35
Enter marks of student 12: 57
Enter marks of student 13: 59
Enter marks of student 14: 71
Enter marks of student 15: 72
Enter marks of student 16: 15
Enter marks of student 17: 61
Enter marks of student 18: 73
Enter marks of student 19: 16
Enter marks of student 20: 61
Number of students passed: 17
Number of students failed: 3

```

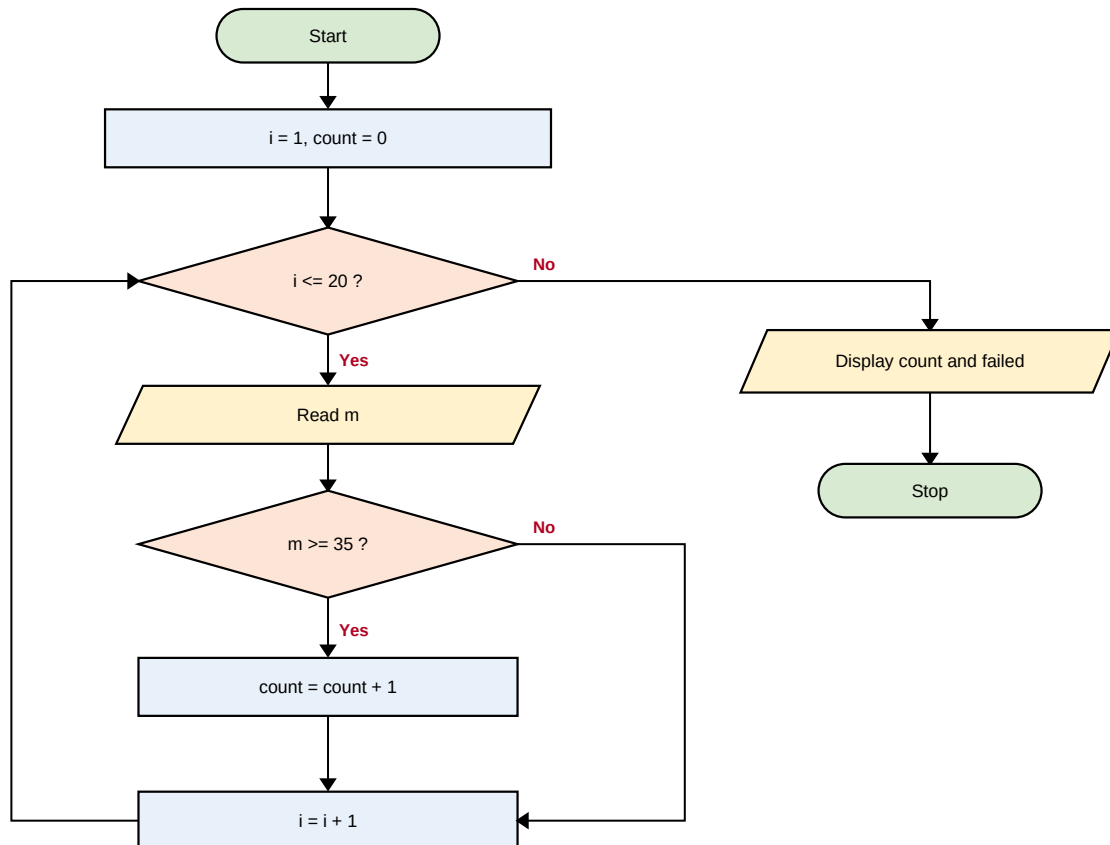
Answer 73 - Set 3 of 10 (Pass mark 35, while loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (35). Each time the condition is True the counter **count** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **count = 0** and the loop variable ($i = 1$).
3. Repeat steps 4 to 6 while $i \leq 20$ (once for each of the 20 students).
4. Read the marks of the student into **m**.
5. If $m \geq 35$, the student has passed, so add 1: **count = count + 1**.
6. Increase the loop variable: $i = i + 1$.
7. Display the number of students passed (**count**) and failed ($20 - \text{count}$).
8. Stop.

Flowchart



Program

```
count = 0
i = 1
while i <= 20:
    m = int(input("Marks of student " + str(i) + ": "))
    if m >= 35:
        count = count + 1
    i = i + 1
print("Number of students passed:", count)
print("Number of students failed:", 20 - count)
```

Sample Input / Output (typed values appear after each prompt)

```
Marks of student 1: 78
Marks of student 2: 60
Marks of student 3: 35
Marks of student 4: 94
Marks of student 5: 59
Marks of student 6: 35
Marks of student 7: 68
Marks of student 8: 19
Marks of student 9: 96
```

```

Marks of student 10: 62
Marks of student 11: 78
Marks of student 12: 23
Marks of student 13: 45
Marks of student 14: 54
Marks of student 15: 39
Marks of student 16: 96
Marks of student 17: 27
Marks of student 18: 98
Marks of student 19: 55
Marks of student 20: 82
Number of students passed: 17
Number of students failed: 3

```

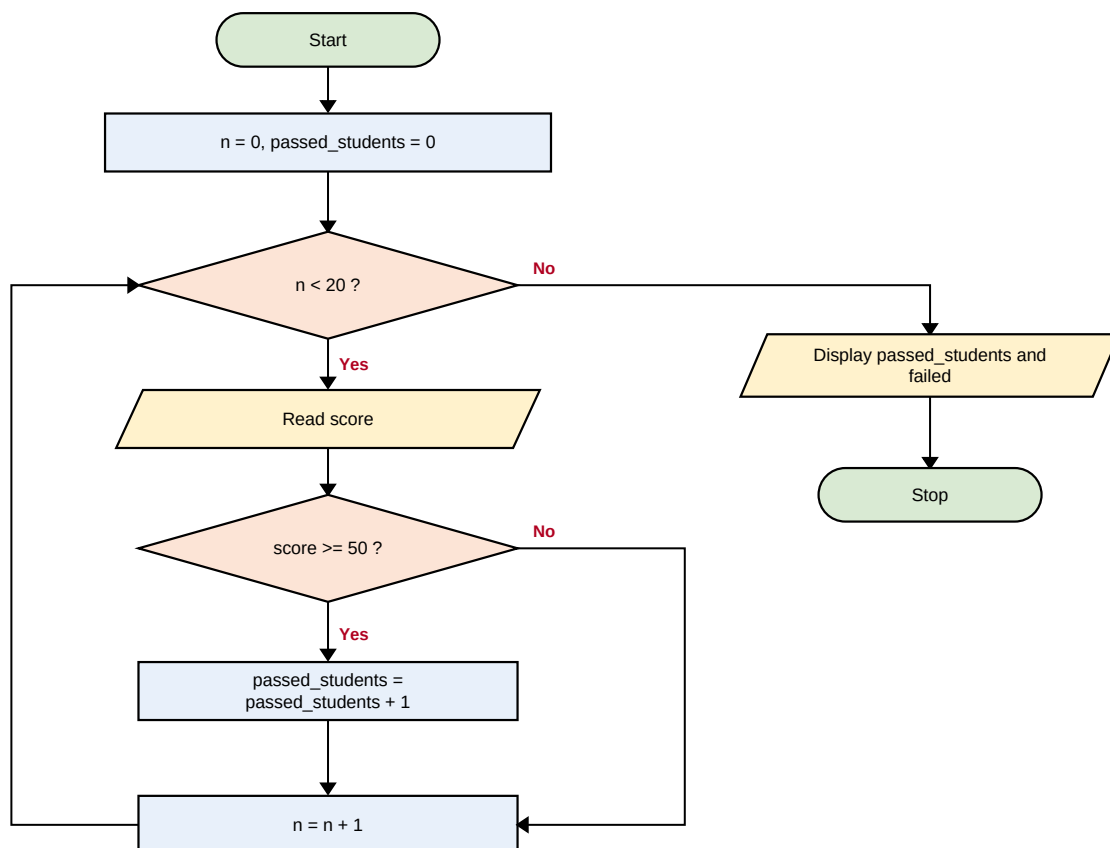
Answer 74 - Set 4 of 10 (Pass mark 50, while loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (50). Each time the condition is True the counter **passed_students** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **passed_students = 0** and the loop variable ($n = 0$).
3. Repeat steps 4 to 6 while $n < 20$ (once for each of the 20 students).
4. Read the marks of the student into **score**.
5. If **score** ≥ 50 , the student has passed, so add 1: **passed_students = passed_students + 1**.
6. Increase the loop variable: $n = n + 1$.
7. Display the number of students passed (**passed_students**) and failed ($20 - \text{passed_students}$).
8. Stop.

Flowchart



Program

```
passed_students = 0
failed = 0
n = 0
while n < 20:
    score = int(input(f"Enter marks of student {n + 1}: "))
    if score >= 50:
        passed_students = passed_students + 1
    else:
        failed += 1
    n = n + 1
print("Number of students passed:", passed_students)
print("Number of students failed:", failed)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter marks of student 1: 35
Enter marks of student 2: 33
Enter marks of student 3: 59
Enter marks of student 4: 38
Enter marks of student 5: 62
Enter marks of student 6: 53
Enter marks of student 7: 31
Enter marks of student 8: 96
Enter marks of student 9: 16
Enter marks of student 10: 19
Enter marks of student 11: 68
Enter marks of student 12: 29
Enter marks of student 13: 17
Enter marks of student 14: 18
Enter marks of student 15: 85
Enter marks of student 16: 73
Enter marks of student 17: 67
Enter marks of student 18: 18
Enter marks of student 19: 25
Enter marks of student 20: 96
Number of students passed: 9
Number of students failed: 11
```

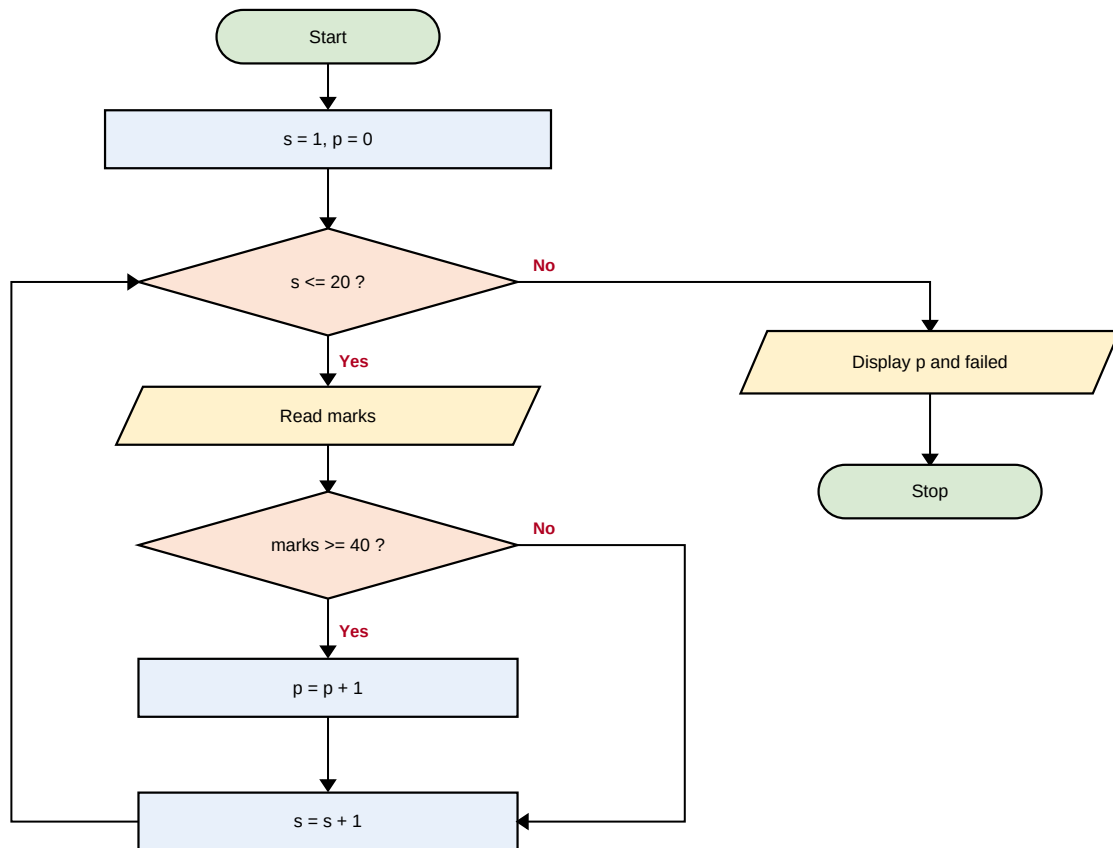
Answer 75 - Set 5 of 10 (Pass mark 40, for loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (40). Each time the condition is True the counter **p** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **p = 0** and the loop variable (**s = 1**).
3. Repeat steps 4 to 6 while **s <= 20** (once for each of the 20 students).
4. Read the marks of the student into **marks**.
5. If **marks >= 40**, the student has passed, so add 1: **p = p + 1**.
6. Increase the loop variable: **s = s + 1**.
7. Display the number of students passed (**p**) and failed (20 - p).
8. Stop.

Flowchart



Program

```

p = 0
for s in range(1, 21):
    marks = int(input("Enter marks of student " + str(s) + ": "))
    if marks >= 40:
        p = p + 1
print("Number of students passed:", p)
print("Number of students failed:", 20 - p)

```

Sample Input / Output (typed values appear after each prompt)

```

Enter marks of student 1: 73
Enter marks of student 2: 87
Enter marks of student 3: 60
Enter marks of student 4: 15
Enter marks of student 5: 38
Enter marks of student 6: 20
Enter marks of student 7: 33
Enter marks of student 8: 37
Enter marks of student 9: 61
Enter marks of student 10: 78
Enter marks of student 11: 92
Enter marks of student 12: 87
Enter marks of student 13: 83
Enter marks of student 14: 52
Enter marks of student 15: 27
Enter marks of student 16: 56
Enter marks of student 17: 18
Enter marks of student 18: 44
Enter marks of student 19: 66
Enter marks of student 20: 53
Number of students passed: 13
Number of students failed: 7

```

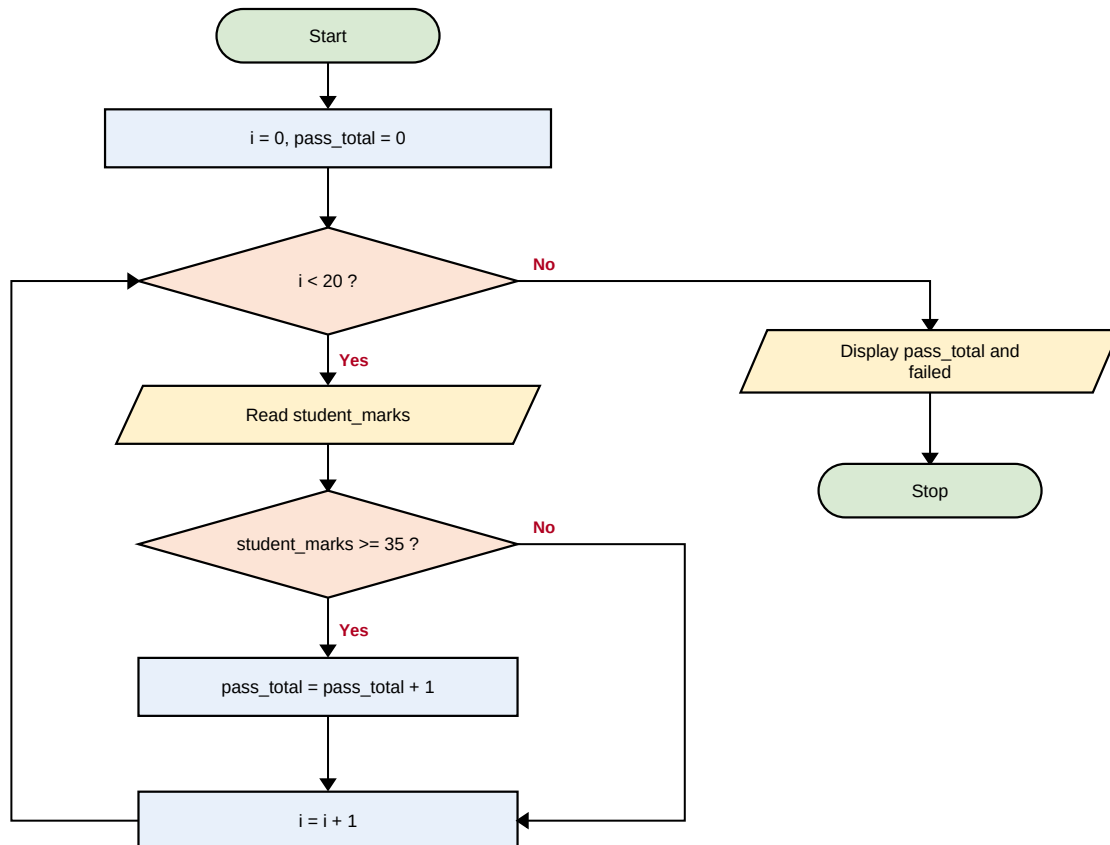
Answer 76 - Set 6 of 10 (Pass mark 35, for loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (35). Each time the condition is True the counter **pass_total** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **pass_total = 0** and the loop variable (**i = 0**).
3. Repeat steps 4 to 6 while **i < 20** (once for each of the 20 students).
4. Read the marks of the student into **student_marks**.
5. If **student_marks >= 35**, the student has passed, so add 1: **pass_total = pass_total + 1**.
6. Increase the loop variable: **i = i + 1**.
7. Display the number of students passed (**pass_total**) and failed (**20 - pass_total**).
8. Stop.

Flowchart



Program

```
pass_total = 0
for i in range(20):
    student_marks = int(input(f"Enter marks of student {i + 1}: "))
    if student_marks >= 35:
        pass_total = pass_total + 1
print("Number of students passed:", pass_total)
print("Number of students failed:", 20 - pass_total)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter marks of student 1: 33
Enter marks of student 2: 46
Enter marks of student 3: 52
Enter marks of student 4: 49
Enter marks of student 5: 57
Enter marks of student 6: 74
Enter marks of student 7: 51
Enter marks of student 8: 82
Enter marks of student 9: 82
Enter marks of student 10: 96
Enter marks of student 11: 58
```

```

Enter marks of student 12: 21
Enter marks of student 13: 66
Enter marks of student 14: 92
Enter marks of student 15: 65
Enter marks of student 16: 89
Enter marks of student 17: 82
Enter marks of student 18: 97
Enter marks of student 19: 26
Enter marks of student 20: 56
Number of students passed: 17
Number of students failed: 3

```

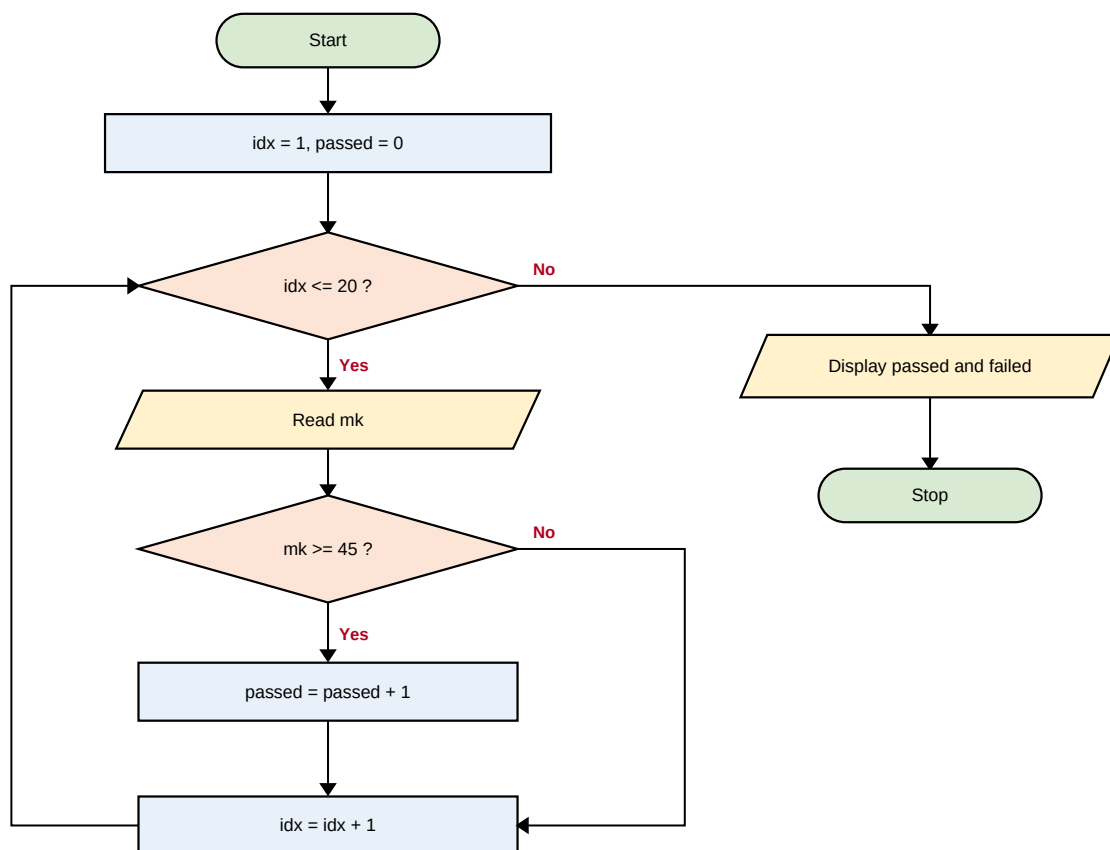
Answer 77 - Set 7 of 10 (Pass mark 45, while loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (45). Each time the condition is True the counter **passed** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **passed = 0** and the loop variable (**idx = 1**).
3. Repeat steps 4 to 6 while **idx <= 20** (once for each of the 20 students).
4. Read the marks of the student into **mk**.
5. If **mk >= 45**, the student has passed, so add 1: **passed = passed + 1**.
6. Increase the loop variable: **idx = idx + 1**.
7. Display the number of students passed (**passed**) and failed (**20 - passed**).
8. Stop.

Flowchart



Program

```
passed = 0
failed = 0
idx = 1
while idx <= 20:
    mk = int(input("Marks of student " + str(idx) + ": "))
    if mk >= 45:
        passed = passed + 1
    else:
        failed += 1
    idx = idx + 1
print("Number of students passed:", passed)
print("Number of students failed:", failed)
```

Sample Input / Output (typed values appear after each prompt)

```
Marks of student 1: 76
Marks of student 2: 48
Marks of student 3: 63
Marks of student 4: 87
Marks of student 5: 67
Marks of student 6: 48
Marks of student 7: 44
Marks of student 8: 81
Marks of student 9: 36
Marks of student 10: 68
Marks of student 11: 58
Marks of student 12: 76
Marks of student 13: 32
Marks of student 14: 27
Marks of student 15: 17
Marks of student 16: 97
Marks of student 17: 62
Marks of student 18: 30
Marks of student 19: 19
Marks of student 20: 93
Number of students passed: 13
Number of students failed: 7
```

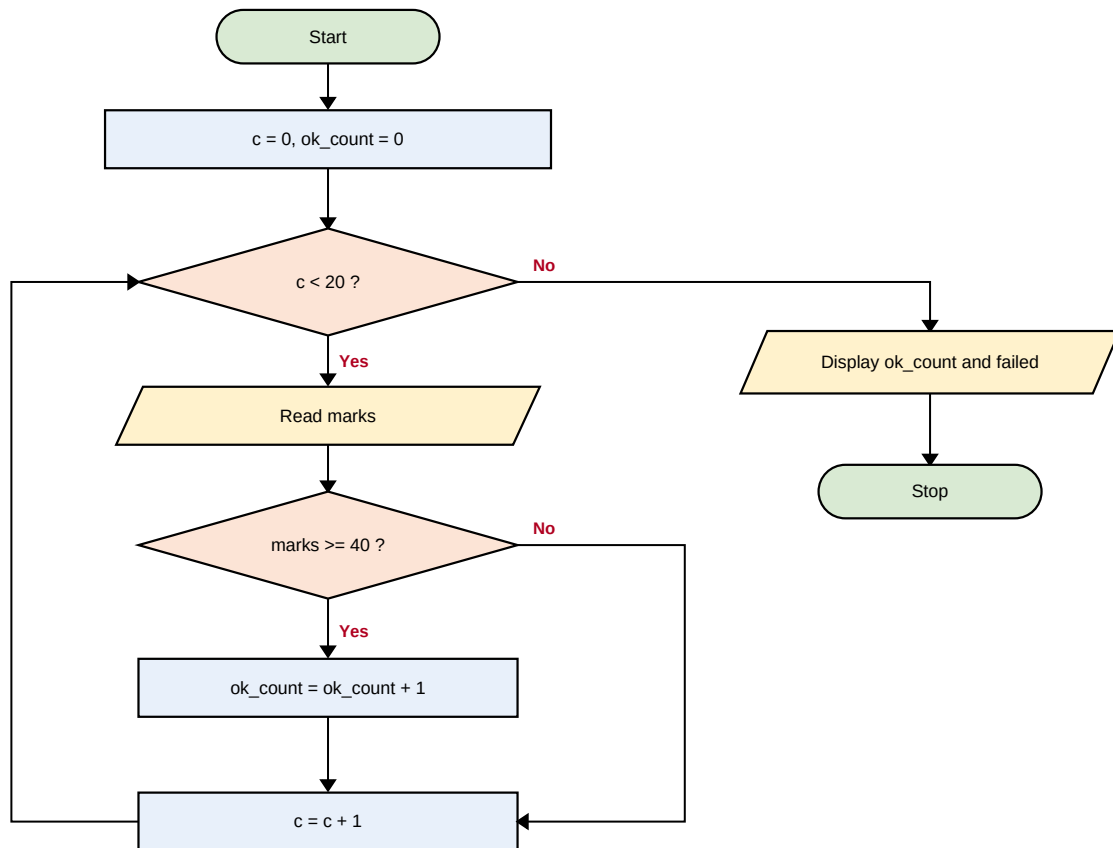
Answer 78 - Set 8 of 10 (Pass mark 40, while loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (40). Each time the condition is True the counter **ok_count** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **ok_count = 0** and the loop variable (**c = 0**).
3. Repeat steps 4 to 6 while **c < 20** (once for each of the 20 students).
4. Read the marks of the student into **marks**.
5. If **marks >= 40**, the student has passed, so add 1: **ok_count = ok_count + 1**.
6. Increase the loop variable: **c = c + 1**.
7. Display the number of students passed (**ok_count**) and failed (**20 - ok_count**).
8. Stop.

Flowchart



Program

```

ok_count = 0
c = 0
while c < 20:
    marks = int(input(f"Enter marks of student {c + 1}: "))
    if marks >= 40:
        ok_count = ok_count + 1
    c = c + 1
print("Number of students passed:", ok_count)
print("Number of students failed:", 20 - ok_count)

```

Sample Input / Output (typed values appear after each prompt)

```

Enter marks of student 1: 75
Enter marks of student 2: 93
Enter marks of student 3: 87
Enter marks of student 4: 32
Enter marks of student 5: 76
Enter marks of student 6: 97
Enter marks of student 7: 45
Enter marks of student 8: 71
Enter marks of student 9: 60
Enter marks of student 10: 97
Enter marks of student 11: 27
Enter marks of student 12: 61
Enter marks of student 13: 40
Enter marks of student 14: 29
Enter marks of student 15: 70
Enter marks of student 16: 34
Enter marks of student 17: 16
Enter marks of student 18: 31
Enter marks of student 19: 38
Enter marks of student 20: 83
Number of students passed: 13
Number of students failed: 7

```

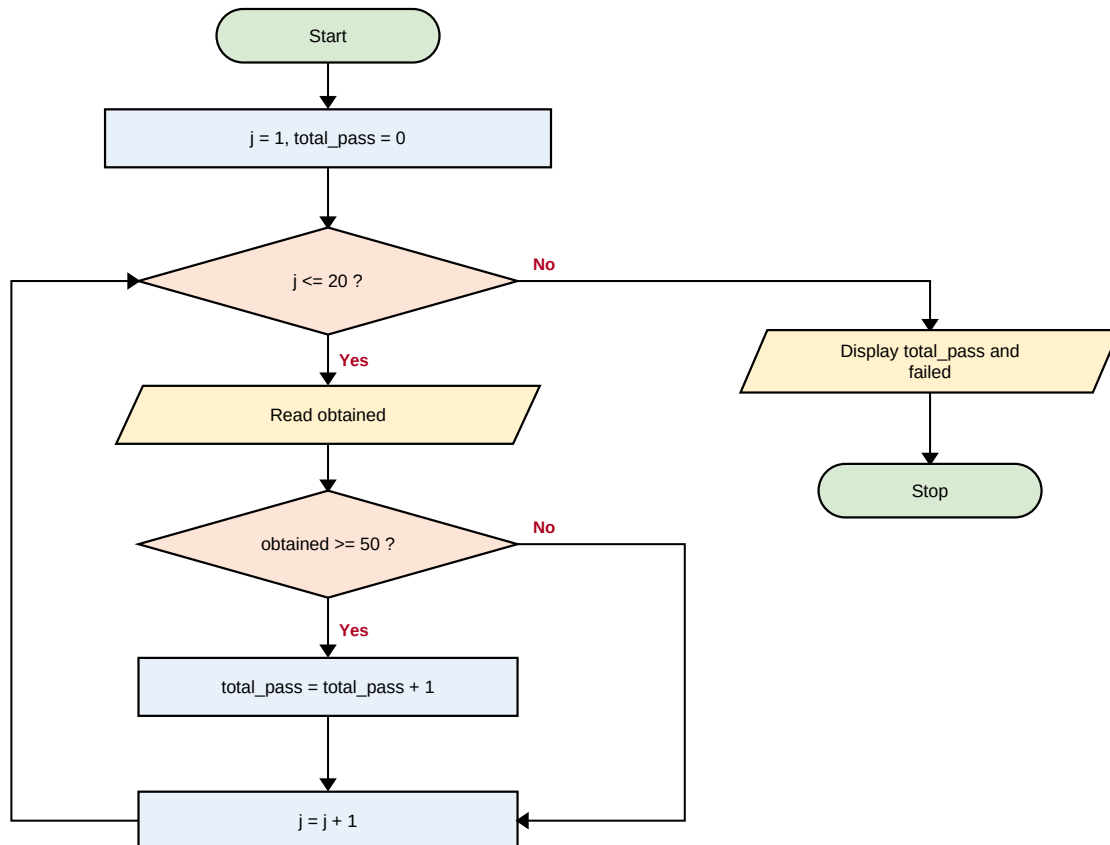
Answer 79 - Set 9 of 10 (Pass mark 50, for loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (50). Each time the condition is True the counter **total_pass** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **total_pass = 0** and the loop variable ($j = 1$).
3. Repeat steps 4 to 6 while $j \leq 20$ (once for each of the 20 students).
4. Read the marks of the student into **obtained**.
5. If **obtained** ≥ 50 , the student has passed, so add 1: **total_pass = total_pass + 1**.
6. Increase the loop variable: $j = j + 1$.
7. Display the number of students passed (**total_pass**) and failed ($20 - \text{total_pass}$).
8. Stop.

Flowchart



Program

```
total_pass = 0
for j in range(1, 21):
    obtained = int(input("Enter marks of student " + str(j) + ": "))
    if obtained >= 50:
        total_pass = total_pass + 1
print("Number of students passed:", total_pass)
print("Number of students failed:", 20 - total_pass)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter marks of student 1: 83
Enter marks of student 2: 98
Enter marks of student 3: 67
Enter marks of student 4: 77
Enter marks of student 5: 22
Enter marks of student 6: 26
Enter marks of student 7: 47
Enter marks of student 8: 60
Enter marks of student 9: 24
Enter marks of student 10: 95
Enter marks of student 11: 81
```

```

Enter marks of student 12: 61
Enter marks of student 13: 96
Enter marks of student 14: 76
Enter marks of student 15: 86
Enter marks of student 16: 90
Enter marks of student 17: 86
Enter marks of student 18: 58
Enter marks of student 19: 25
Enter marks of student 20: 86
Number of students passed: 15
Number of students failed: 5

```

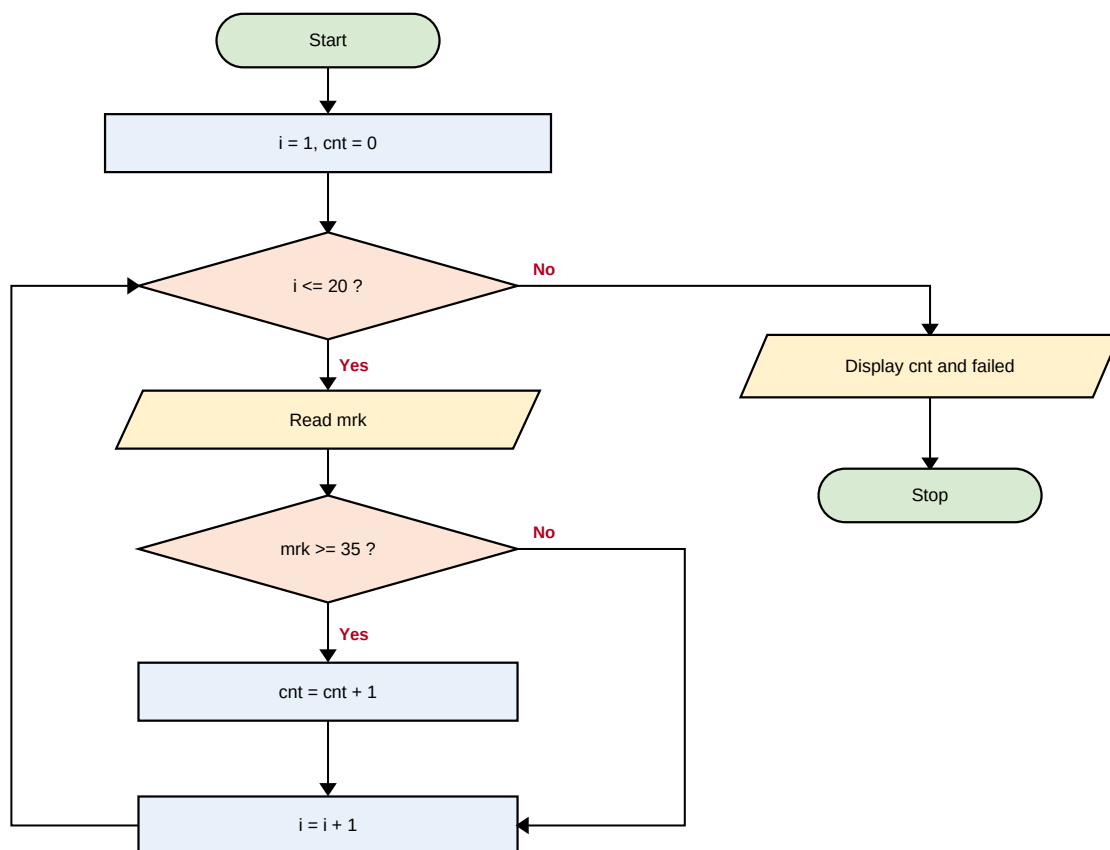
Answer 80 - Set 10 of 10 (Pass mark 35, while loop)

A **loop** repeats the input step for all 20 students and a **decision-making statement (if)** checks whether the marks are at least the pass mark (35). Each time the condition is True the counter **cnt** is increased by one.

Algorithm

1. Start.
2. Set the pass counter **cnt = 0** and the loop variable (**i = 1**).
3. Repeat steps 4 to 6 while **i <= 20** (once for each of the 20 students).
4. Read the marks of the student into **mrk**.
5. If **mrk >= 35**, the student has passed, so add 1: **cnt = cnt + 1**.
6. Increase the loop variable: **i = i + 1**.
7. Display the number of students passed (**cnt**) and failed (**20 - cnt**).
8. Stop.

Flowchart



Program

```
cnt = 0
failed = 0
i = 1
while i <= 20:
    mrk = int(input("Marks of student " + str(i) + ": "))
    if mrk >= 35:
        cnt = cnt + 1
    else:
        failed += 1
    i = i + 1
print("Number of students passed:", cnt)
print("Number of students failed:", failed)
```

Sample Input / Output (typed values appear after each prompt)

```
Marks of student 1: 67
Marks of student 2: 40
Marks of student 3: 35
Marks of student 4: 31
Marks of student 5: 29
Marks of student 6: 88
Marks of student 7: 48
Marks of student 8: 85
Marks of student 9: 98
Marks of student 10: 98
Marks of student 11: 95
Marks of student 12: 53
Marks of student 13: 63
Marks of student 14: 53
Marks of student 15: 20
Marks of student 16: 89
Marks of student 17: 49
Marks of student 18: 16
Marks of student 19: 80
Marks of student 20: 54
Number of students passed: 16
Number of students failed: 4
```

Section 9 : Group 3 (Roll Number ends in 2, 4, 9) - Question 3 (Answers 81 - 90)

Question: Explain the purpose of `input()` and `print()` functions in Python with examples.

Answer 81 - Set 1 of 10 (Name and age greeting)

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with `int()` or `float()` when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *input() returns a string, so age is converted with int() before the calculation.*

Program

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Hello", name)
print("Next year you will be", age + 1, "years old")
```

Sample Input / Output (typed values appear after each prompt)

```
Enter your name: Rohit
Enter your age: 19
Hello Rohit
Next year you will be 20 years old
```

Answer 82 - Set 2 of 10 (Sum and product of two numbers)

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with `int()` or `float()` when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *print() can display several values separated by commas; a space is placed between them.*

Program

```
first = int(input("Enter first number: "))
second = int(input("Enter second number: "))
print("Sum =", first + second)
print("Product =", first * second)
```

Sample Input / Output (typed values appear after each prompt)

```
Enter first number: 14
Enter second number: 6
Sum = 20
Product = 84
```

Answer 83 - Set 3 of 10 (print() with sep and end)

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with `int()` or `float()` when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *sep changes the separator between values and end changes what is printed at the end (default is a new line).*

Program

```
city = input("Enter your city: ")
print("Welcome", "to", city, sep="-")
print("Python", end=" | ")
print("is", end=" | ")
print("fun")
```

Sample Input / Output (typed values appear after each prompt)

```
Enter your city: Mysuru
Welcome-to-Mysuru
Python | is | fun
```

Answer 84 - Set 4 of 10 (Area of a circle (float input))

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with **int()** or **float()** when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *float()* converts the typed text to a decimal number; the f-string formats the answer to 2 decimals.

Program

```
radius = float(input("Enter radius: "))
area = 3.14159 * radius * radius
print(f"Area of circle with radius {radius} = {area:.2f}")
```

Sample Input / Output (typed values appear after each prompt)

```
Enter radius: 4.5
Area of circle with radius 4.5 = 63.62
```

Answer 85 - Set 5 of 10 (Reading two values in one line)

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with **int()** or **float()** when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *split()* breaks the input line into separate strings; they are text until converted with **int()**.

Program

```
x, y = input("Enter two numbers separated by space: ").split()
print("x =", x, "y =", y)
print("Concatenated:", x + y)
print("Added:", int(x) + int(y))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter two numbers separated by space: 12 30
x = 12 y = 30
Concatenated: 1230
Added: 42
```

Answer 86 - Set 6 of 10 (Bill using format())

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with **int()** or **float()** when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *str.format()* fills the {} placeholders with the given values in order.

Program

```
item = input("Enter item name: ")
price = float(input("Enter price: "))
qty = int(input("Enter quantity: "))
print("Item: {}, Price: {:.2f}, Quantity: {}".format(item, price, qty))
print("Total = {:.2f}".format(price * qty))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter item name: Notebook
Enter price: 45.5
Enter quantity: 6
Item: Notebook, Price: 45.50, Quantity: 6
Total = 273.00
```

Answer 87 - Set 7 of 10 (Escape sequences in print())

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with **int()** or **float()** when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *It prints a tab and \n starts a new line inside the printed text.*

Program

```
subject = input("Enter subject: ")
marks = input("Enter marks: ")
print("Subject\tMarks")
print(subject + "\t" + marks)
print("Line one\nLine two")
```

Sample Input / Output (typed values appear after each prompt)

```
Enter subject: Maths
Enter marks: 92
SubjectMarks
Maths92
Line one
Line two
```

Answer 88 - Set 8 of 10 (% formatting)

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with **int()** or **float()** when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *%s, %d and %.1f are format specifiers for a string, an integer and a decimal number.*

Program

```
student = input("Enter student name: ")
score = int(input("Enter score out of 150: "))
print("%s scored %d marks" % (student, score))
print("Percentage: %.1f%%" % (score / 150 * 100))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter student name: Divya
Enter score out of 150: 120
Divya scored 120 marks
Percentage: 80.0%
```

Answer 89 - Set 9 of 10 (List of numbers from one line)

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with **int()** or **float()** when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *map(int, ...) converts every piece of the input line to an integer.*

Program

```
numbers = list(map(int, input("Enter numbers separated by space: ").split()))
print("Numbers:", numbers)
print("Total:", sum(numbers))
print("Largest:", max(numbers))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter numbers separated by space: 8 15 3 27 11
Numbers: [8, 15, 3, 27, 11]
Total: 64
Largest: 27
```

Answer 90 - Set 10 of 10 (Temperature conversion)

input() reads a line typed by the user from the keyboard. An optional prompt message is shown first and the value is always returned as a **string**, so it is converted with **int()** or **float()** when a number is needed. **print()** sends output to the screen. It can print many values, and its **sep** and **end** arguments control the separator and the line ending. *end="" keeps the next print() on the same line.*

Program

```
temp = float(input("Enter temperature in Celsius: "))
fahrenheit = round(temp * 9 / 5 + 32, 1)
print("Celsius:", temp, end="")
print(" C =", fahrenheit, "F")
print("Type of input value:", type(temp))
```

Sample Input / Output (typed values appear after each prompt)

```
Enter temperature in Celsius: 25.5
Celsius: 25.5 C = 77.9 F
Type of input value: <class 'float'>
```